

Complete Course: Computer Vision

From Pixels to Deep Learning

Data Science and Machine Learning Mastery

Contents

1	Introduction to Computer Vision	3
1.1	The Human Visual System vs Computer Vision	3
1.2	Applications of Computer Vision	3
2	Fundamentals of Digital Images	3
2.1	What is an Image?	3
2.2	Image Representation	4
2.2.1	Grayscale Images	4
2.2.2	Color Images	4
2.2.3	Other Color Spaces	5
2.3	Image Properties and Metrics	5
3	Image Processing Fundamentals	8
3.1	Basic Operations	8
3.1.1	Arithmetic Operations	8
3.1.2	Logical Operations	8
3.1.3	Geometric Transformations	9
3.2	Color Space Conversions	12
4	Image Filtering and Enhancement	15
4.1	Image Histograms and Contrast	15
4.2	Histogram Equalization	16
4.3	Spatial Filtering	16
4.3.1	Convolution Operation	16
4.3.2	Common Filters	17
5	Feature Detection and Description	23
5.1	Corner Detection	23
5.2	Edge Detection	23
5.3	Feature Descriptors	24

6	Image Segmentation	31
6.1	Thresholding	31
6.2	GrabCut Algorithm	32
7	Convolutional Neural Networks (CNNs)	40
7.1	The Convolution Operation in Neural Networks	40
7.2	CNN Architecture Components	41
7.3	Famous CNN Architectures	41
8	Advanced Computer Vision Topics	55
8.1	Object Detection	55
8.2	Image Segmentation (Advanced)	55
8.3	GANs for Computer Vision	56
9	Computer Vision Applications	69
9.1	Face Recognition	69
9.2	Optical Character Recognition (OCR)	69
9.3	Medical Imaging	69
10	Best Practices and Future Directions	70
10.1	Best Practices in Computer Vision	70
10.2	Emerging Trends	70
10.3	Resources and Tools	71
11	Conclusion	71
11.1	Next Steps	71
11.2	Final Advice	72

1 Introduction to Computer Vision

What is Computer Vision?

Computer Vision is the field of teaching computers to "see" - to extract meaningful information from images and videos, and to understand the visual world the way humans do.

1.1 The Human Visual System vs Computer Vision

Human Visual System	Computer Vision
Eyes: Capture light	Camera: Captures pixels
Retina: Pre-processes	Sensors: Convert to digital
Brain: Processes patterns	Algorithms: Extract features
Experience: Learns from examples	Training: Learns from labeled data
Real-time: Parallel processing	GPU: Parallel computation

Table 1: Human vs Computer Vision comparison

1.2 Applications of Computer Vision

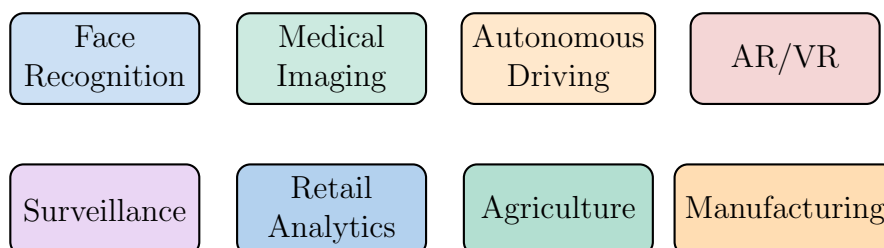


Figure 1: Diverse applications of computer vision

2 Fundamentals of Digital Images

2.1 What is an Image?

Definition 2.1 (Digital Image). *A digital image is a 2D array of pixels (picture elements), where each pixel represents color and intensity at a specific location.*

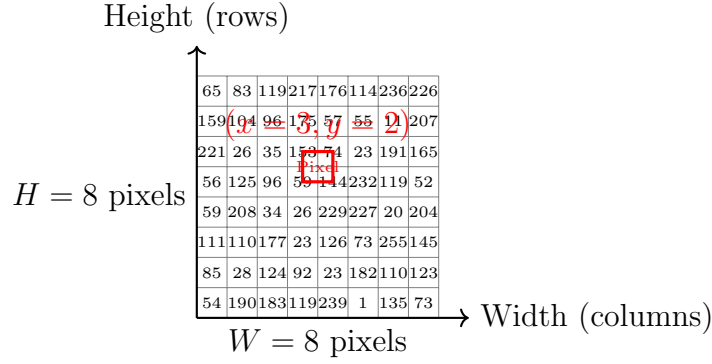


Figure 2: A digital image as a 2D grid of pixels

2.2 Image Representation

2.2.1 Grayscale Images

Definition 2.2 (Grayscale Image). *Single channel image where each pixel represents intensity from 0 (black) to 255 (white).*

$$I(x, y) \in [0, 255]$$

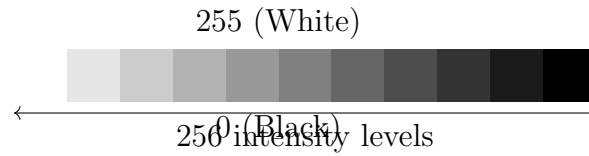


Figure 3: Grayscale intensity range

2.2.2 Color Images

Definition 2.3 (RGB Color Model). *Color images use 3 channels: Red, Green, Blue. Each pixel is a tuple (R, G, B) where each value $\in [0, 255]$.*

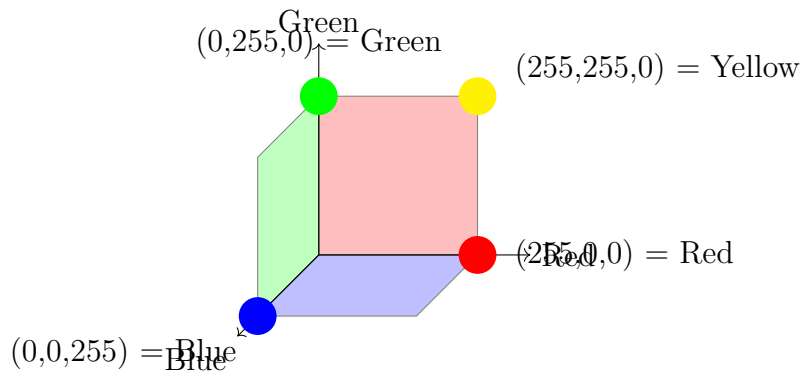


Figure 4: RGB color space cube

Example 2.4 (Common color representations). • *Black:* $(0, 0, 0)$

- *White:* $(255, 255, 255)$
- *Gray:* $(128, 128, 128)$
- *Red:* $(255, 0, 0)$
- *Green:* $(0, 255, 0)$
- *Blue:* $(0, 0, 255)$
- *Yellow:* $(255, 255, 0)$
- *Magenta:* $(255, 0, 255)$
- *Cyan:* $(0, 255, 255)$

2.2.3 Other Color Spaces

Color Space	Components	Use Case
RGB	Red, Green, Blue	Display, Storage
HSV/HSB	Hue, Saturation, Value/Lightness	Color segmentation
LAB	Lightness, A (green-red), B (blue-yellow)	Color difference
YUV/YCbCr	Luminance, Chrominance	Video compression
CMYK	Cyan, Magenta, Yellow, Key (black)	Printing

Table 2: Different color spaces in computer vision

2.3 Image Properties and Metrics

Definition 2.5 (Image Resolution). *Number of pixels in an image: $Resolution = Width \times Height$*

Definition 2.6 (Aspect Ratio). *Ratio of width to height: $Aspect\ Ratio = \frac{Width}{Height}$*

Definition 2.7 (Bit Depth). *Number of bits used to represent each pixel's color.*

- *8-bit grayscale:* 256 shades (0-255)
- *24-bit color:* 8 bits per channel (16.7 million colors)
- *32-bit:* 24-bit color + 8-bit alpha (transparency)

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Create a simple image from scratch
6 # Grayscale image (8-bit)
7 height, width = 100, 150
8 gray_image = np.random.randint(0, 256, (height, width), dtype=np.uint8)
9
10 # Color image (RGB)
11 color_image = np.zeros((height, width, 3), dtype=np.uint8)
12 color_image[:, :, 0] = 255 # Red channel
13 color_image[30:70, 50:100, 1] = 255 # Green square
14 color_image[50:90, 80:130, 2] = 255 # Blue square
15
16 # Display images
17 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
18
19 axes[0].imshow(gray_image, cmap='gray')
20 axes[0].set_title(f'Grayscale Image: {height}x{width}')
21 axes[0].set_xlabel('Width (columns)')
22 axes[0].set_ylabel('Height (rows)')
23
24 axes[1].imshow(color_image)
25 axes[1].set_title(f'Color Image: {height}x{width}x3')
26 axes[1].set_xlabel('Width')
27 axes[1].set_ylabel('Height')
28
29 plt.tight_layout()
30 plt.show()
31
32 # Access pixel values
33 print(f"Grayscale pixel at (50, 75): {gray_image[50, 75]}")
34 print(f"Color pixel at (50, 75): {color_image[50, 75]}")
35
36 # Image properties
37 print(f"\nGrayscale image shape: {gray_image.shape}")
38 print(f"Color image shape: {color_image.shape}")
39 print(f"Image dtype: {gray_image.dtype}")
40 print(f"Total pixels (grayscale): {gray_image.size}")
41 print(f"Total pixels (color): {color_image.size}")
42 print(f"Memory size (grayscale): {gray_image.nbytes} bytes")
43 print(f"Memory size (color): {color_image.nbytes} bytes")
44
45 # Create gradient images
46 # Horizontal gradient
47 x_gradient = np.tile(np.arange(width), (height, 1)).astype(np.uint8)

```

```

48
49 # Vertical gradient
50 y_gradient = np.tile(np.arange(height).reshape(-1, 1), (1, width)).astype(np.
    ↪ uint8)
51
52 # Color gradient
53 color_gradient = np.zeros((100, 100, 3), dtype=np.uint8)
54 for i in range(100):
55     color_gradient[i, :, 0] = i * 2.55 # Red gradient
56     color_gradient[:, i, 1] = i * 2.55 # Green gradient
57
58 plt.figure(figsize=(15, 5))
59 plt.subplot(131)
60 plt.imshow(x_gradient, cmap='gray')
61 plt.title('Horizontal Gradient')
62
63 plt.subplot(132)
64 plt.imshow(y_gradient, cmap='gray')
65 plt.title('Vertical Gradient')
66
67 plt.subplot(133)
68 plt.imshow(color_gradient)
69 plt.title('Color Gradient')
70
71 plt.tight_layout()
72 plt.show()

```

Listing 1: Basic Image Operations with OpenCV

3 Image Processing Fundamentals

3.1 Basic Operations

3.1.1 Arithmetic Operations

Operation	Mathematical	Purpose
Addition	$I_{out}(x, y) = I_1(x, y) + I_2(x, y)$	Combine images, Add effects
Subtraction	$I_{out}(x, y) = I_1(x, y) - I_2(x, y) $	Change detection, Background removal
Multiplication	$I_{out}(x, y) = I(x, y) \times k$	Brightness adjustment
Division	$I_{out}(x, y) = I(x, y)/k$	Darkening, Contrast adjustment
Blending	$I_{out} = \alpha I_1 + (1 - \alpha) I_2$	Image compositing

Table 3: Basic arithmetic operations on images

3.1.2 Logical Operations

Operation	Mathematical	Purpose
AND	$I_{out} = I_1 \& I_2$	Masking, Region extraction
OR	$I_{out} = I_1 I_2$	Combining regions
XOR	$I_{out} = I_1 \oplus I_2$	Difference detection
NOT	$I_{out} = \sim I$	Inversion, Negative image

Table 4: Logical operations on binary images

3.1.3 Geometric Transformations

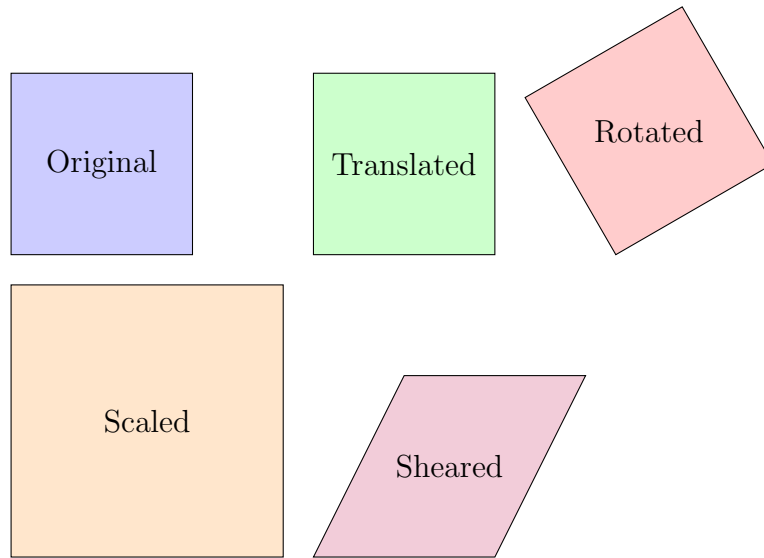


Figure 5: Geometric transformations on images

Definition 3.1 (Affine Transformation). *Combination of linear transformations (rotation, scaling, shearing) and translation:*

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Create sample images
6 # Image 1: Gradient
7 img1 = np.zeros((100, 100), dtype=np.uint8)
8 for i in range(100):
9     img1[:, i] = i * 2.55
10
11 # Image 2: Circle
12 img2 = np.zeros((100, 100), dtype=np.uint8)
13 cv2.circle(img2, (50, 50), 40, 255, -1)
14
15 # Arithmetic operations
16 add_img = cv2.add(img1, img2)
17 subtract_img = cv2.subtract(img1, img2)
18 multiply_img = cv2.multiply(img1, 0.5) # Darken
19 blend_img = cv2.addWeighted(img1, 0.7, img2, 0.3, 0)
20
```

```

21 # Logical operations (convert to binary)
22 _, binary1 = cv2.threshold(img1, 127, 255, cv2.THRESH_BINARY)
23 _, binary2 = cv2.threshold(img2, 127, 255, cv2.THRESH_BINARY)
24
25 and_img = cv2.bitwise_and(binary1, binary2)
26 or_img = cv2.bitwise_or(binary1, binary2)
27 xor_img = cv2.bitwise_xor(binary1, binary2)
28 not_img = cv2.bitwise_not(binary1)
29
30 # Display results
31 fig, axes = plt.subplots(3, 4, figsize=(15, 10))
32
33 images = [
34     ('Original 1', img1, 'gray'),
35     ('Original 2', img2, 'gray'),
36     ('Addition', add_img, 'gray'),
37     ('Subtraction', subtract_img, 'gray'),
38     ('Multiplication', multiply_img, 'gray'),
39     ('Blending', blend_img, 'gray'),
40     ('Binary 1', binary1, 'gray'),
41     ('Binary 2', binary2, 'gray'),
42     ('AND', and_img, 'gray'),
43     ('OR', or_img, 'gray'),
44     ('XOR', xor_img, 'gray'),
45     ('NOT', not_img, 'gray')
46 ]
47
48 for idx, (title, img, cmap) in enumerate(images):
49     row, col = divmod(idx, 4)
50     axes[row, col].imshow(img, cmap=cmap)
51     axes[row, col].set_title(title)
52     axes[row, col].axis('off')
53
54 plt.tight_layout()
55 plt.show()
56
57 # Geometric transformations
58 # Load a real image (or create one)
59 real_img = np.zeros((100, 100, 3), dtype=np.uint8)
60 cv2.rectangle(real_img, (20, 20), (80, 80), (255, 0, 0), -1)
61 cv2.circle(real_img, (50, 50), 30, (0, 255, 0), -1)
62
63 # Translation
64 M_translation = np.float32([[1, 0, 25], [0, 1, 25]]) # Move 25 pixels right and
    ↪ down
65 translated = cv2.warpAffine(real_img, M_translation, (100, 100))
66

```

```

67 # Rotation
68 center = (50, 50)
69 angle = 45
70 scale = 1.0
71 M_rotation = cv2.getRotationMatrix2D(center, angle, scale)
72 rotated = cv2.warpAffine(real_img, M_rotation, (100, 100))
73
74 # Scaling
75 scaled = cv2.resize(real_img, None, fx=1.5, fy=1.5, interpolation=cv2.
    ↪ INTER_LINEAR)
76
77 # Affine transformation
78 pts1 = np.float32([[20, 20], [80, 20], [20, 80]])
79 pts2 = np.float32([[10, 30], [70, 20], [20, 90]])
80 M_affine = cv2.getAffineTransform(pts1, pts2)
81 affine = cv2.warpAffine(real_img, M_affine, (100, 100))
82
83 # Perspective transformation
84 pts1 = np.float32([[20, 20], [80, 20], [20, 80], [80, 80]])
85 pts2 = np.float32([[10, 30], [70, 20], [20, 90], [80, 70]])
86 M_perspective = cv2.getPerspectiveTransform(pts1, pts2)
87 perspective = cv2.warpPerspective(real_img, M_perspective, (100, 100))
88
89 # Display geometric transformations
90 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
91
92 geo_images = [
93     ('Original', real_img),
94     ('Translated', translated),
95     ('Rotated 45degrees', rotated),
96     ('Scaled 1.5x', scaled),
97     ('Affine', affine),
98     ('Perspective', perspective)
99 ]
100
101 for idx, (title, img) in enumerate(geo_images):
102     row, col = divmod(idx, 3)
103     axes[row, col].imshow(img)
104     axes[row, col].set_title(title)
105     axes[row, col].axis('off')
106
107 plt.tight_layout()
108 plt.show()
109
110 # Image statistics
111 print("Image Statistics:")
112 print(f"Min pixel value: {np.min(real_img)}")

```

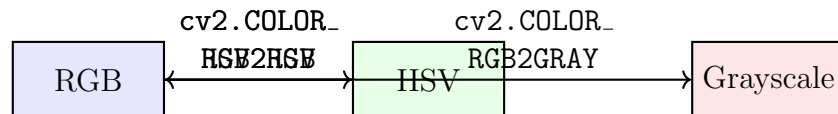
```

113 print(f"Max pixel value: {np.max(real_img)}")
114 print(f"Mean pixel value: {np.mean(real_img):.2f}")
115 print(f"Standard deviation: {np.std(real_img):.2f}")
116 print(f"Histogram bins: {np.histogram(real_img.ravel(), bins=256)[0].shape}")

```

Listing 2: Image Operations with OpenCV

3.2 Color Space Conversions



HSV Components:

Hue: Color type (0-180°)
 Saturation: Color intensity (0-255)
 Value: Brightness (0-255)

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Create a colorful test image
6 colorful_img = np.zeros((100, 100, 3), dtype=np.uint8)
7
8 # Add different colors
9 colorful_img[0:33, :, 0] = 255 # Red band
10 colorful_img[33:66, :, 1] = 255 # Green band
11 colorful_img[66:100, :, 2] = 255 # Blue band
12
13 # Add a yellow circle
14 cv2.circle(colorful_img, (50, 50), 30, (255, 255, 0), -1)
15
16 # Convert to different color spaces
17 # Note: OpenCV uses BGR by default, but we're using RGB for display
18 rgb_img = colorful_img.copy()
19
20 # Convert RGB to HSV
21 hsv_img = cv2.cvtColor(rgb_img, cv2.COLOR_RGB2HSV)
22
23 # Convert RGB to LAB
24 lab_img = cv2.cvtColor(rgb_img, cv2.COLOR_RGB2LAB)
25
26 # Convert RGB to YCrCb (YUV)
27 yuv_img = cv2.cvtColor(rgb_img, cv2.COLOR_RGB2YCrCb)

```

```

28
29 # Convert to grayscale
30 gray_img = cv2.cvtColor(rgb_img, cv2.COLOR_RGB2GRAY)
31
32 # Display all color spaces
33 fig, axes = plt.subplots(2, 4, figsize=(16, 8))
34
35 # RGB channels
36 axes[0, 0].imshow(rgb_img)
37 axes[0, 0].set_title('RGB Image')
38 axes[0, 0].axis('off')
39
40 # RGB channels separately
41 rgb_titles = ['Red Channel', 'Green Channel', 'Blue Channel']
42 for i in range(3):
43     axes[0, i+1].imshow(rgb_img[:, :, i], cmap='gray')
44     axes[0, i+1].set_title(rgb_titles[i])
45     axes[0, i+1].axis('off')
46
47 # HSV channels
48 hsv_titles = ['Hue', 'Saturation', 'Value']
49 for i in range(3):
50     axes[1, i].imshow(hsv_img[:, :, i], cmap='gray')
51     axes[1, i].set_title(f'HSV - {hsv_titles[i]}')
52     axes[1, i].axis('off')
53
54 # Grayscale
55 axes[1, 3].imshow(gray_img, cmap='gray')
56 axes[1, 3].set_title('Grayscale')
57 axes[1, 3].axis('off')
58
59 plt.tight_layout()
60 plt.show()
61
62 # Color segmentation example using HSV
63 # Create a range for red color in HSV
64 # Red has two ranges in HSV due to wrap-around at 180degrees
65 lower_red1 = np.array([0, 100, 100])
66 upper_red1 = np.array([10, 255, 255])
67 lower_red2 = np.array([160, 100, 100])
68 upper_red2 = np.array([180, 255, 255])
69
70 # Create masks
71 mask1 = cv2.inRange(hsv_img, lower_red1, upper_red1)
72 mask2 = cv2.inRange(hsv_img, lower_red2, upper_red2)
73 red_mask = cv2.bitwise_or(mask1, mask2)
74

```

```

75 # Apply mask to original image
76 red_segmented = cv2.bitwise_and(rgb_img, rgb_img, mask=red_mask)
77
78 # Green segmentation
79 lower_green = np.array([40, 100, 100])
80 upper_green = np.array([80, 255, 255])
81 green_mask = cv2.inRange(hsv_img, lower_green, upper_green)
82 green_segmented = cv2.bitwise_and(rgb_img, rgb_img, mask=green_mask)
83
84 # Blue segmentation
85 lower_blue = np.array([100, 100, 100])
86 upper_blue = np.array([140, 255, 255])
87 blue_mask = cv2.inRange(hsv_img, lower_blue, upper_blue)
88 blue_segmented = cv2.bitwise_and(rgb_img, rgb_img, mask=blue_mask)
89
90 # Display color segmentation
91 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
92
93 segmentation_images = [
94     ('Original', rgb_img),
95     ('Red Mask', red_mask),
96     ('Red Segmented', red_segmented),
97     ('Green Mask', green_mask),
98     ('Green Segmented', green_segmented),
99     ('Blue Mask', blue_mask),
100    ('Blue Segmented', blue_segmented)
101 ]
102
103 for idx, (title, img) in enumerate(segmentation_images[:6]):
104     row, col = divmod(idx, 3)
105     if 'Mask' in title:
106         axes[row, col].imshow(img, cmap='gray')
107     else:
108         axes[row, col].imshow(img)
109         axes[row, col].set_title(title)
110         axes[row, col].axis('off')
111
112 plt.tight_layout()
113 plt.show()
114
115 # Color histogram analysis
116 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
117
118 # RGB histogram
119 colors = ('r', 'g', 'b')
120 for i, color in enumerate(colors):
121     histogram = cv2.calcHist([rgb_img], [i], None, [256], [0, 256])

```

```

122     axes[0, 0].plot(histogram, color=color)
123 axes[0, 0].set_title('RGB Histogram')
124 axes[0, 0].set_xlabel('Pixel Value')
125 axes[0, 0].set_ylabel('Frequency')
126 axes[0, 0].legend(['Red', 'Green', 'Blue'])
127 axes[0, 0].grid(True, alpha=0.3)
128
129 # Grayscale histogram
130 axes[0, 1].hist(gray_img.ravel(), 256, [0, 256])
131 axes[0, 1].set_title('Grayscale Histogram')
132 axes[0, 1].set_xlabel('Pixel Value')
133 axes[0, 1].set_ylabel('Frequency')
134 axes[0, 1].grid(True, alpha=0.3)
135
136 # HSV histograms
137 hsv_colors = ('Hue', 'Saturation', 'Value')
138 for i in range(3):
139     histogram = cv2.calcHist([hsv_img], [i], None, [256], [0, 256])
140     axes[1, 0].plot(histogram, label=hsv_colors[i])
141 axes[1, 0].set_title('HSV Histogram')
142 axes[1, 0].set_xlabel('Pixel Value')
143 axes[1, 0].set_ylabel('Frequency')
144 axes[1, 0].legend()
145 axes[1, 0].grid(True, alpha=0.3)
146
147 # Cumulative histogram
148 cumulative_hist = np.cumsum(cv2.calcHist([gray_img], [0], None, [256], [0, 256]))
149 axes[1, 1].plot(cumulative_hist)
150 axes[1, 1].set_title('Cumulative Histogram')
151 axes[1, 1].set_xlabel('Pixel Value')
152 axes[1, 1].set_ylabel('Cumulative Frequency')
153 axes[1, 1].grid(True, alpha=0.3)
154
155 plt.tight_layout()
156 plt.show()

```

Listing 3: Color Space Conversions

4 Image Filtering and Enhancement

4.1 Image Histograms and Contrast

Definition 4.1 (Image Histogram). *A histogram represents the distribution of pixel intensities in an image. For grayscale images, it shows how many pixels have each intensity value (0-255).*

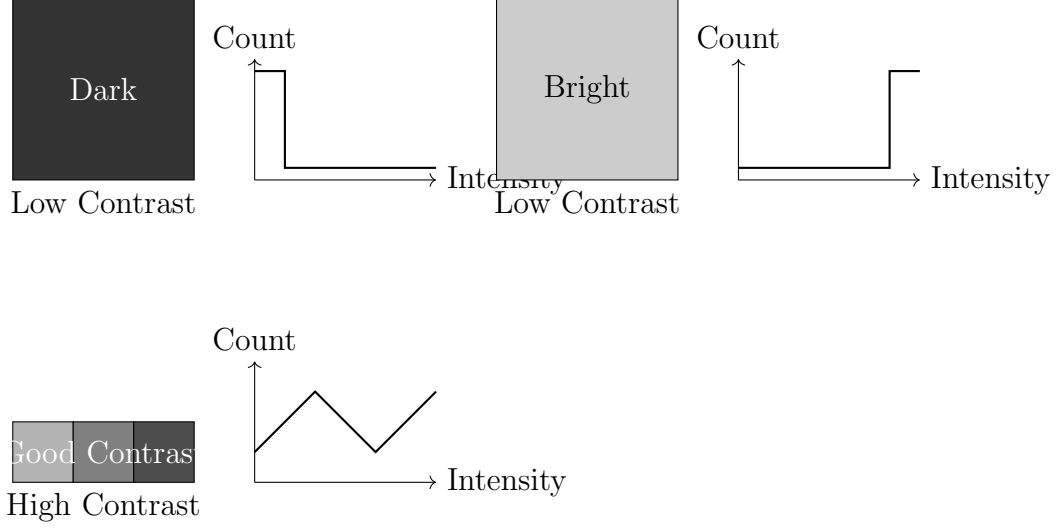


Figure 6: Image contrast and corresponding histograms

4.2 Histogram Equalization

Definition 4.2 (Histogram Equalization). *A method to improve contrast by spreading out intensity values across the full range.*

$$s_k = T(r_k) = \sum_{j=0}^k \frac{n_j}{n} \times (L - 1)$$

where r_k is input intensity, s_k is output intensity, n_j is count of pixels with intensity j , n is total pixels, L is number of intensity levels (256).

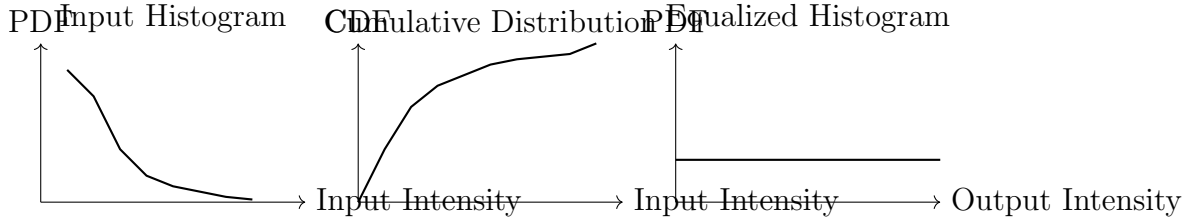


Figure 7: Histogram equalization process

4.3 Spatial Filtering

4.3.1 Convolution Operation

Definition 4.3 (Convolution). *In image processing, convolution is applying a kernel (small matrix) to an image to produce a new image:*

$$I_{out}(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k K(i, j) \cdot I(x + i, y + j)$$

where K is the kernel of size $(2k + 1) \times (2k + 1)$.

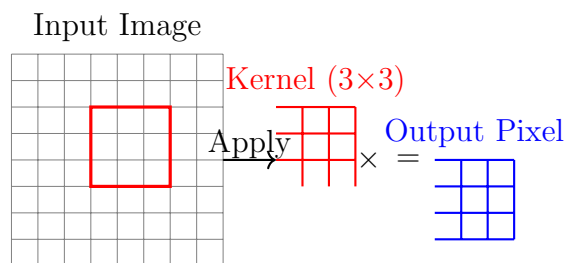


Figure 8: Convolution operation: kernel slides over image

4.3.2 Common Filters

Filter Type	Kernel	Purpose
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	No change
Box Blur	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	Simple blurring
Gaussian Blur	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	Smoothing, noise reduction
Sobel X	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	Horizontal edge detection
Sobel Y	$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$	Vertical edge detection
Laplacian	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	Edge detection
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	Enhance edges

Table 5: Common convolution filters

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy import ndimage

```

```

5
6 # Create a test image with different features
7 test_img = np.zeros((200, 200), dtype=np.uint8)
8
9 # Add different patterns
10 # Gradient
11 for i in range(200):
12     test_img[:, i] = i * 1.27
13
14 # Add circles
15 cv2.circle(test_img, (50, 50), 30, 200, -1)
16 cv2.circle(test_img, (150, 50), 30, 100, -1)
17
18 # Add squares
19 test_img[100:140, 30:70] = 150
20 test_img[100:140, 130:170] = 50
21
22 # Add noise
23 noise = np.random.normal(0, 20, test_img.shape).astype(np.uint8)
24 noisy_img = cv2.add(test_img, noise)
25
26 # 1. Histogram Equalization
27 # Global histogram equalization
28 equalized = cv2.equalizeHist(noisy_img)
29
30 # CLAHE (Contrast Limited Adaptive Histogram Equalization)
31 clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
32 clahe_img = clahe.apply(noisy_img)
33
34 # 2. Spatial Filtering
35 # Define kernels
36 identity_kernel = np.array([[0, 0, 0],
37                             [0, 1, 0],
38                             [0, 0, 0]])
39
40 box_blur_kernel = np.ones((3,3), np.float32) / 9
41
42 gaussian_kernel = np.array([[1, 2, 1],
43                             [2, 4, 2],
44                             [1, 2, 1]], np.float32) / 16
45
46 sobel_x_kernel = np.array([[ -1, 0, 1],
47                             [ -2, 0, 2],
48                             [ -1, 0, 1]])
49
50 sobel_y_kernel = np.array([[ -1, -2, -1],
51                             [ 0, 0, 0],

```

```

52         [1, 2, 1]))
53
54 laplacian_kernel = np.array([[0, 1, 0],
55                             [1, -4, 1],
56                             [0, 1, 0]])
57
58 sharpen_kernel = np.array([[0, -1, 0],
59                             [-1, 5, -1],
60                             [0, -1, 0]])
61
62 # Apply filters
63 identity = cv2.filter2D(noisy_img, -1, identity_kernel)
64 box_blur = cv2.filter2D(noisy_img, -1, box_blur_kernel)
65 gaussian_blur = cv2.filter2D(noisy_img, -1, gaussian_kernel)
66 sobel_x = cv2.filter2D(noisy_img, cv2.CV_64F, sobel_x_kernel)
67 sobel_y = cv2.filter2D(noisy_img, cv2.CV_64F, sobel_y_kernel)
68 laplacian = cv2.filter2D(noisy_img, cv2.CV_64F, laplacian_kernel)
69 sharpened = cv2.filter2D(noisy_img, -1, sharpen_kernel)
70
71 # Normalize edge detection results for display
72 sobel_x_norm = cv2.normalize(sobel_x, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)
73 sobel_y_norm = cv2.normalize(sobel_y, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)
74 laplacian_norm = cv2.normalize(laplacian, None, 0, 255, cv2.NORM_MINMAX, cv2.
    ↪ CV_8U)
75
76 # 3. Built-in OpenCV functions
77 gaussian_blur_cv = cv2.GaussianBlur(noisy_img, (5,5), 0)
78 median_blur = cv2.medianBlur(noisy_img, 5)
79 bilateral_filter = cv2.bilateralFilter(noisy_img, 9, 75, 75)
80
81 # Display results
82 fig, axes = plt.subplots(4, 4, figsize=(16, 16))
83
84 images = [
85     ('Original', test_img, 'gray'),
86     ('Noisy', noisy_img, 'gray'),
87     ('Hist Eq', equalized, 'gray'),
88     ('CLAHE', clahe_img, 'gray'),
89     ('Box Blur', box_blur, 'gray'),
90     ('Gaussian Blur', gaussian_blur, 'gray'),
91     ('Median Blur', median_blur, 'gray'),
92     ('Bilateral', bilateral_filter, 'gray'),
93     ('Sobel X', sobel_x_norm, 'gray'),
94     ('Sobel Y', sobel_y_norm, 'gray'),
95     ('Laplacian', laplacian_norm, 'gray'),
96     ('Sharpened', sharpened, 'gray'),
97     ('Gaussian CV', gaussian_blur_cv, 'gray'),

```

```

98     ('Sobel X Abs', np.abs(sobel_x), 'gray'),
99     ('Sobel Y Abs', np.abs(sobel_y), 'gray'),
100    ('Gradient Magnitude', np.sqrt(sobel_x**2 + sobel_y**2), 'gray')
101 ]
102
103 for idx, (title, img, cmap) in enumerate(images):
104     row, col = divmod(idx, 4)
105     axes[row, col].imshow(img, cmap=cmap)
106     axes[row, col].set_title(title)
107     axes[row, col].axis('off')
108
109 plt.tight_layout()
110 plt.show()
111
112 # Display histograms
113 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
114
115 hist_images = [test_img, noisy_img, equalized, clahe_img, box_blur, gaussian_blur
116               ↪ ]
117 hist_titles = ['Original', 'Noisy', 'Hist Eq', 'CLAHE', 'Box Blur', 'Gaussian
118               ↪ Blur']
119
120 for idx, (img, title) in enumerate(zip(hist_images, hist_titles)):
121     row, col = divmod(idx, 3)
122     axes[row, col].hist(img.ravel(), 256, [0, 256])
123     axes[row, col].set_title(f'{title} Histogram')
124     axes[row, col].set_xlabel('Pixel Value')
125     axes[row, col].set_ylabel('Frequency')
126     axes[row, col].grid(True, alpha=0.3)
127
128 plt.tight_layout()
129 plt.show()
130
131 # Frequency domain filtering (FFT)
132 # Perform FFT
133 f = np.fft.fft2(noisy_img)
134 fshift = np.fft.fftshift(f)
135 magnitude_spectrum = 20*np.log(np.abs(fshift))
136
137 # Create filters
138 rows, cols = noisy_img.shape
139 crow, ccol = rows//2, cols//2
140
141 # Low-pass filter (circular mask)
142 mask_low = np.zeros((rows, cols), np.uint8)
143 r = 30
144 cv2.circle(mask_low, (ccol, crow), r, 1, -1)

```

```

143
144 # High-pass filter
145 mask_high = np.ones((rows, cols), np.uint8)
146 cv2.circle(mask_high, (ccol, crow), r, 0, -1)
147
148 # Band-pass filter (ring)
149 mask_band = np.zeros((rows, cols), np.uint8)
150 r_inner, r_outer = 20, 50
151 cv2.circle(mask_band, (ccol, crow), r_outer, 1, -1)
152 cv2.circle(mask_band, (ccol, crow), r_inner, 0, -1)
153
154 # Apply filters
155 fshift_low = fshift * mask_low
156 fshift_high = fshift * mask_high
157 fshift_band = fshift * mask_band
158
159 # Inverse FFT
160 f_ishift_low = np.fft.ifftshift(fshift_low)
161 img_low = np.fft.ifft2(f_ishift_low)
162 img_low = np.abs(img_low)
163
164 f_ishift_high = np.fft.ifftshift(fshift_high)
165 img_high = np.fft.ifft2(f_ishift_high)
166 img_high = np.abs(img_high)
167
168 f_ishift_band = np.fft.ifftshift(fshift_band)
169 img_band = np.fft.ifft2(f_ishift_band)
170 img_band = np.abs(img_band)
171
172 # Display frequency domain results
173 fig, axes = plt.subplots(2, 4, figsize=(16, 8))
174
175 freq_images = [
176     ('Original', noisy_img, 'gray'),
177     ('FFT Magnitude', magnitude_spectrum, 'gray'),
178     ('Low-pass Mask', mask_low * 255, 'gray'),
179     ('Low-pass Result', img_low, 'gray'),
180     ('High-pass Mask', mask_high * 255, 'gray'),
181     ('High-pass Result', img_high, 'gray'),
182     ('Band-pass Mask', mask_band * 255, 'gray'),
183     ('Band-pass Result', img_band, 'gray')
184 ]
185
186 for idx, (title, img, cmap) in enumerate(freq_images):
187     row, col = divmod(idx, 4)
188     axes[row, col].imshow(img, cmap=cmap)
189     axes[row, col].set_title(title)

```

```

190     axes[row, col].axis('off')
191
192 plt.tight_layout()
193 plt.show()
194
195 # Edge detection comparison
196 canny_edges = cv2.Canny(noisy_img, 100, 200)
197
198 # Morphological operations for edge enhancement
199 kernel_morph = np.ones((3,3), np.uint8)
200 eroded = cv2.erode(noisy_img, kernel_morph, iterations=1)
201 dilated = cv2.dilate(noisy_img, kernel_morph, iterations=1)
202 opened = cv2.morphologyEx(noisy_img, cv2.MORPH_OPEN, kernel_morph)
203 closed = cv2.morphologyEx(noisy_img, cv2.MORPH_CLOSE, kernel_morph)
204 gradient = cv2.morphologyEx(noisy_img, cv2.MORPH_GRADIENT, kernel_morph)
205
206 # Display edge detection and morphological operations
207 fig, axes = plt.subplots(2, 4, figsize=(16, 8))
208
209 morph_images = [
210     ('Canny Edges', canny_edges, 'gray'),
211     ('Eroded', eroded, 'gray'),
212     ('Dilated', dilated, 'gray'),
213     ('Opened', opened, 'gray'),
214     ('Closed', closed, 'gray'),
215     ('Gradient', gradient, 'gray'),
216     ('Top Hat', cv2.morphologyEx(noisy_img, cv2.MORPH_TOPHAT, kernel_morph), 'gray
        ↪ '),
217     ('Black Hat', cv2.morphologyEx(noisy_img, cv2.MORPH_BLACKHAT, kernel_morph), '
        ↪ gray')
218 ]
219
220 for idx, (title, img, cmap) in enumerate(morph_images):
221     row, col = divmod(idx, 4)
222     axes[row, col].imshow(img, cmap=cmap)
223     axes[row, col].set_title(title)
224     axes[row, col].axis('off')
225
226 plt.tight_layout()
227 plt.show()

```

Listing 4: Image Filtering and Enhancement

5 Feature Detection and Description

5.1 Corner Detection

Definition 5.1 (Harris Corner Detector). *Detects corners by analyzing intensity changes in all directions. A corner is a point where intensity changes significantly in multiple directions.*

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

where I_x and I_y are image derivatives, and $w(x,y)$ is a window function.

Corner response: $R = \det(M) - k \cdot \text{trace}(M)^2$

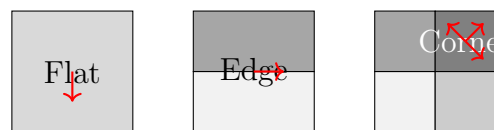
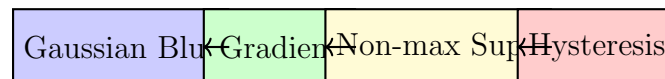


Figure 9: Corner detection: corners have intensity changes in all directions

5.2 Edge Detection

Definition 5.2 (Canny Edge Detector). *Multi-stage algorithm:*

1. **Noise reduction** with Gaussian filter
2. **Gradient calculation** using Sobel filters
3. **Non-maximum suppression** to thin edges
4. **Hysteresis thresholding** with two thresholds



Strong Edge: $\text{gradient} > T_{\text{high}}$ **Weak Edge:** $T_{\text{low}} < \text{gradient} \leq T_{\text{high}}$ **No Edge:** $\text{gradient} < T_{\text{low}}$

Figure 10: Canny edge detector pipeline

5.3 Feature Descriptors

Descriptor	Description	Advantages
SIFT	Scale-Invariant Feature Transform	Rotation and scale invariant
SURF	Speeded-Up Robust Features	Faster than SIFT
ORB	Oriented FAST and Rotated BRIEF	Fast, binary descriptor
BRIEF	Binary Robust Independent Features	Compact, fast matching
HOG	Histogram of Oriented Gradients	Good for object detection

Table 6: Feature descriptors in computer vision

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Create a test image with different features
6 test_img = np.zeros((300, 400), dtype=np.uint8)
7
8 # Add geometric shapes
9 # Rectangle
10 cv2.rectangle(test_img, (50, 50), (150, 150), 200, -1)
11 # Circle
12 cv2.circle(test_img, (300, 100), 50, 150, -1)
13 # Triangle
14 pts = np.array([[200, 200], [250, 250], [150, 250]], np.int32)
15 cv2.fillPoly(test_img, [pts], 100)
16
17 # Add texture with noise
18 texture = np.random.normal(128, 30, (100, 100)).astype(np.uint8)
19 test_img[200:300, 300:400] = texture
20
21 # Add checkerboard pattern
22 for i in range(0, 100, 20):
23     for j in range(0, 100, 20):
24         if (i//20 + j//20) % 2 == 0:
25             test_img[50+i:50+i+20, 250+j:250+j+20] = 255
26
27 # 1. Corner Detection
28 # Harris Corner Detection
29 harris_img = test_img.copy()
```

```

30 harris_img_color = cv2.cvtColor(harris_img, cv2.COLOR_GRAY2BGR)
31
32 # Harris corner parameters
33 block_size = 2
34 ksize = 3
35 k = 0.04
36
37 # Detect corners
38 dst = cv2.cornerHarris(test_img, block_size, ksize, k)
39 dst = cv2.dilate(dst, None)
40
41 # Threshold for corners
42 harris_img_color[dst > 0.01 * dst.max()] = [0, 0, 255]
43
44 # Shi-Tomasi Corner Detection (Good Features to Track)
45 corners = cv2.goodFeaturesToTrack(test_img, 100, 0.01, 10)
46 corners = np.int0(corners)
47
48 shi_tomasi_img = cv2.cvtColor(test_img, cv2.COLOR_GRAY2BGR)
49 for corner in corners:
50     x, y = corner.ravel()
51     cv2.circle(shi_tomasi_img, (x, y), 3, (0, 255, 0), -1)
52
53 # 2. Edge Detection
54 # Canny Edge Detection
55 canny_low = cv2.Canny(test_img, 50, 150)
56 canny_high = cv2.Canny(test_img, 100, 200)
57 canny_adaptive = cv2.Canny(test_img, 0, 255) # Automatic thresholding
58
59 # Sobel Edges
60 sobel_x = cv2.Sobel(test_img, cv2.CV_64F, 1, 0, ksize=5)
61 sobel_y = cv2.Sobel(test_img, cv2.CV_64F, 0, 1, ksize=5)
62 sobel_magnitude = np.sqrt(sobel_x**2 + sobel_y**2)
63 sobel_magnitude = cv2.normalize(sobel_magnitude, None, 0, 255, cv2.NORM_MINMAX,
    ↪ cv2.CV_8U)
64
65 # Laplacian
66 laplacian = cv2.Laplacian(test_img, cv2.CV_64F)
67 laplacian = cv2.normalize(np.abs(laplacian), None, 0, 255, cv2.NORM_MINMAX, cv2.
    ↪ CV_8U)
68
69 # 3. Feature Descriptors (using ORB as example)
70 # Initialize ORB detector
71 orb = cv2.ORB_create(nfeatures=100)
72
73 # Find keypoints and descriptors
74 keypoints_orb, descriptors_orb = orb.detectAndCompute(test_img, None)

```

```

75
76 # Draw keypoints
77 orb_img = cv2.drawKeypoints(test_img, keypoints_orb, None, color=(0, 255, 0),
    ↪ flags=0)
78
79 # SIFT (requires opencv-contrib-python)
80 try:
81     sift = cv2.SIFT_create()
82     keypoints_sift, descriptors_sift = sift.detectAndCompute(test_img, None)
83     sift_img = cv2.drawKeypoints(test_img, keypoints_sift, None,
84                                   color=(255, 0, 0), flags=cv2.
    ↪ DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
85 except:
86     sift_img = test_img.copy()
87     print("SIFT not available (requires opencv-contrib-python)")
88
89 # Display results
90 fig, axes = plt.subplots(3, 4, figsize=(16, 12))
91
92 images = [
93     ('Original', test_img, 'gray'),
94     ('Harris Corners', harris_img_color, None),
95     ('Shi-Tomasi Corners', shi_tomasi_img, None),
96     ('Canny (50,150)', canny_low, 'gray'),
97     ('Canny (100,200)', canny_high, 'gray'),
98     ('Sobel Magnitude', sobel_magnitude, 'gray'),
99     ('Laplacian', laplacian, 'gray'),
100    ('ORB Keypoints', orb_img, None),
101    ('Canny Adaptive', canny_adaptive, 'gray'),
102    ('Sobel X', cv2.convertScaleAbs(sobel_x), 'gray'),
103    ('Sobel Y', cv2.convertScaleAbs(sobel_y), 'gray'),
104    ('SIFT Keypoints', sift_img if 'sift_img' in locals() else test_img, None)
105 ]
106
107 for idx, (title, img, cmap) in enumerate(images):
108     row, col = divmod(idx, 4)
109     if cmap:
110         axes[row, col].imshow(img, cmap=cmap)
111     else:
112         axes[row, col].imshow(img)
113         axes[row, col].set_title(title)
114         axes[row, col].axis('off')
115
116 plt.tight_layout()
117 plt.show()
118
119 # 4. Blob Detection

```

```

120 # Simple blob detection using Laplacian of Gaussian
121 blob_img = test_img.copy()
122
123 # Detect blobs using simple method
124 # Apply Gaussian blur
125 blurred = cv2.GaussianBlur(blob_img, (11, 11), 0)
126 # Laplacian
127 laplacian_blob = cv2.Laplacian(blurred, cv2.CV_64F)
128 # Find zero crossings (simple blob centers)
129 laplacian_blob_abs = np.abs(laplacian_blob)
130
131 # Threshold for blob centers
132 _, blob_centers = cv2.threshold(laplacian_blob_abs.astype(np.uint8), 50, 255, cv2.
    ↪ .THRESH_BINARY)
133
134 # Find contours of blobs
135 contours, _ = cv2.findContours(blob_centers, cv2.RETR_EXTERNAL, cv2.
    ↪ CHAIN_APPROX_SIMPLE)
136
137 # Draw blob centers
138 blob_result = cv2.cvtColor(blob_img, cv2.COLOR_GRAY2BGR)
139 for contour in contours:
140     # Calculate moments
141     M = cv2.moments(contour)
142     if M['m00'] != 0:
143         # Calculate centroid
144         cx = int(M['m10'] / M['m00'])
145         cy = int(M['m01'] / M['m00'])
146         cv2.circle(blob_result, (cx, cy), 5, (0, 255, 0), -1)
147
148 # 5. Contour Detection
149 # Find contours in the original image
150 _, binary = cv2.threshold(test_img, 127, 255, cv2.THRESH_BINARY)
151 contours, hierarchy = cv2.findContours(binary, cv2.RETR_TREE, cv2.
    ↪ CHAIN_APPROX_SIMPLE)
152
153 # Draw contours
154 contour_img = cv2.cvtColor(test_img, cv2.COLOR_GRAY2BGR)
155 cv2.drawContours(contour_img, contours, -1, (0, 255, 0), 2)
156
157 # Calculate contour properties
158 contour_properties = []
159 for i, contour in enumerate(contours[:5]): # First 5 contours
160     area = cv2.contourArea(contour)
161     perimeter = cv2.arcLength(contour, True)
162
163     # Approximate contour

```

```

164     epsilon = 0.02 * perimeter
165     approx = cv2.approxPolyDP(contour, epsilon, True)
166
167     # Bounding box
168     x, y, w, h = cv2.boundingRect(contour)
169
170     # Aspect ratio
171     aspect_ratio = float(w) / h if h != 0 else 0
172
173     # Extent
174     rect_area = w * h
175     extent = float(area) / rect_area if rect_area != 0 else 0
176
177     contour_properties.append({
178         'contour': i,
179         'area': area,
180         'perimeter': perimeter,
181         'vertices': len(approx),
182         'aspect_ratio': aspect_ratio,
183         'extent': extent
184     })
185
186 # Display blob and contour detection
187 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
188
189 blob_contour_images = [
190     ('Original', test_img, 'gray'),
191     ('Blob Centers', blob_centers, 'gray'),
192     ('Blob Detection', blob_result, None),
193     ('Binary', binary, 'gray'),
194     ('Contours', contour_img, None),
195     ('Laplacian Blob', laplacian_blob_abs, 'gray')
196 ]
197
198 for idx, (title, img, cmap) in enumerate(blob_contour_images):
199     row, col = divmod(idx, 3)
200     if cmap:
201         axes[row, col].imshow(img, cmap=cmap)
202     else:
203         axes[row, col].imshow(img)
204         axes[row, col].set_title(title)
205         axes[row, col].axis('off')
206
207 plt.tight_layout()
208 plt.show()
209
210 # Print contour properties

```

```

211 print("Contour Properties:")
212 print("-" * 80)
213 for props in contour_properties:
214     print(f"Contour {props['contour']}:")
215     print(f" Area: {props['area']:.2f}")
216     print(f" Perimeter: {props['perimeter']:.2f}")
217     print(f" Vertices: {props['vertices']}")
218     print(f" Aspect Ratio: {props['aspect_ratio']:.2f}")
219     print(f" Extent: {props['extent']:.2f}")
220     print()
221
222 # 6. HOG (Histogram of Oriented Gradients)
223 # Calculate HOG features
224 win_size = (64, 64)
225 block_size = (16, 16)
226 block_stride = (8, 8)
227 cell_size = (8, 8)
228 nbins = 9
229
230 # Create HOG descriptor
231 hog = cv2.HOGDescriptor(win_size, block_size, block_stride, cell_size, nbins)
232
233 # Compute HOG features for the entire image
234 hog_features = hog.compute(test_img[:64, :64]) # Use first 64x64 region
235
236 print(f"HOG feature vector shape: {hog_features.shape}")
237 print(f"Number of HOG features: {len(hog_features)}")
238
239 # Visualize HOG
240 hog_image = np.zeros((64, 64), dtype=np.uint8)
241 hog_image[:64, :64] = test_img[:64, :64]
242
243 # Get HOG visualization
244 _, hog_vis = hog.compute(hog_image, visualise=True) if hasattr(hog, 'compute')
    ↪ else (None, None)
245
246 if hog_vis is not None:
247     hog_vis = cv2.normalize(hog_vis, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)
248
249     fig, axes = plt.subplots(1, 2, figsize=(10, 5))
250     axes[0].imshow(hog_image[:64, :64], cmap='gray')
251     axes[0].set_title('Original (64x64)')
252     axes[0].axis('off')
253
254     axes[1].imshow(hog_vis, cmap='gray')
255     axes[1].set_title('HOG Visualization')
256     axes[1].axis('off')

```

```

257     plt.tight_layout()
258     plt.show()
259
260
261 # 7. Feature Matching Example
262 # Create two similar images with slight transformation
263 img1 = test_img.copy()
264 img2 = cv2.rotate(test_img, cv2.ROTATE_90_CLOCKWISE) # Rotate 90 degrees
265
266 # Detect and compute ORB features for both images
267 kp1, desc1 = orb.detectAndCompute(img1, None)
268 kp2, desc2 = orb.detectAndCompute(img2, None)
269
270 # Create BFMatcher
271 bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
272
273 # Match descriptors
274 matches = bf.match(desc1, desc2)
275
276 # Sort matches by distance
277 matches = sorted(matches, key=lambda x: x.distance)
278
279 # Draw matches
280 matching_result = cv2.drawMatches(img1, kp1, img2, kp2, matches[:50], None, flags
    ↪ =2)
281
282 plt.figure(figsize=(15, 8))
283 plt.imshow(matching_result)
284 plt.title('Feature Matching (ORB)')
285 plt.axis('off')
286 plt.show()
287
288 print(f"Number of matches: {len(matches)}")
289 print(f"Best match distance: {matches[0].distance}")
290 print(f"Worst match distance: {matches[-1].distance}")

```

Listing 5: Feature Detection and Description

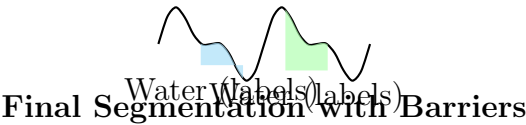
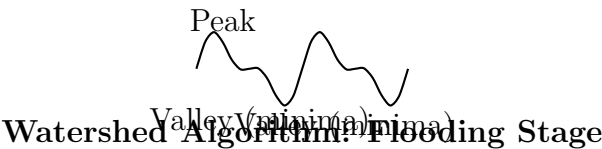
6 Image Segmentation

6.1 Thresholding

Thresholding Type	Description	Mathematical Form
Simple/Binary	Single threshold value	$dst(x,y) = \begin{cases} maxval & \text{if } src(x,y) > thresh \\ 0 & \text{otherwise} \end{cases}$
Adaptive	Local threshold based on neighborhood	$T(x,y) = \mu(x,y) - C$
Otsu's	Automatic threshold selection	Maximizes between-class variance
Multi-level	Multiple thresholds for multiple regions	Multiple threshold values

Table 7: Thresholding methods for image segmentation

Topographic Surface



6.2 GrabCut Algorithm

Algorithm 1 GrabCut Algorithm for Foreground Extraction

```
1: procedure GRABCUT( $I$ , rectangle)
2:   Initialize: Outside rectangle = background, Inside = possible foreground
3:   Build Gaussian Mixture Models (GMM) for foreground and background
4:   repeat
5:     Assign pixels to GMM components
6:     Learn new GMM parameters
7:     Estimate segmentation using graph cut
8:   until convergence or max iterations
9:   return foreground mask
10: end procedure
```

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from skimage import segmentation, filters, color
5 from scipy import ndimage
6
7 # Create a complex test image
8 seg_img = np.zeros((200, 300), dtype=np.uint8)
9
10 # Add different regions with varying intensities
11 # Region 1: Dark circle
12 cv2.circle(seg_img, (75, 75), 40, 50, -1)
13
14 # Region 2: Medium circle
15 cv2.circle(seg_img, (225, 75), 40, 150, -1)
16
17 # Region 3: Bright circle
18 cv2.circle(seg_img, (150, 150), 40, 220, -1)
19
20 # Region 4: Gradient background
21 for i in range(300):
22     seg_img[:, i] = seg_img[:, i] + i//10
23
24 # Add noise
25 noise = np.random.normal(0, 15, seg_img.shape).astype(np.uint8)
26 seg_img = cv2.add(seg_img, noise)
27
28 # 1. Simple Thresholding
29 _, thresh_binary = cv2.threshold(seg_img, 127, 255, cv2.THRESH_BINARY)
30 _, thresh_binary_inv = cv2.threshold(seg_img, 127, 255, cv2.THRESH_BINARY_INV)
31 _, thresh_trunc = cv2.threshold(seg_img, 127, 255, cv2.THRESH_TRUNC)
```

```

32 _, thresh_tozero = cv2.threshold(seg_img, 127, 255, cv2.THRESH_TOZERO)
33 _, thresh_tozero_inv = cv2.threshold(seg_img, 127, 255, cv2.THRESH_TOZERO_INV)
34
35 # 2. Adaptive Thresholding
36 thresh_mean = cv2.adaptiveThreshold(seg_img, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
37                                     cv2.THRESH_BINARY, 11, 2)
38 thresh_gaussian = cv2.adaptiveThreshold(seg_img, 255, cv2.
    ↪ ADAPTIVE_THRESH_GAUSSIAN_C,
39                                         cv2.THRESH_BINARY, 11, 2)
40
41 # 3. Otsu's Thresholding
42 _, thresh_otsu = cv2.threshold(seg_img, 0, 255, cv2.THRESH_BINARY + cv2.
    ↪ THRESH_OTSU)
43
44 # Multi-level Otsu (using skimage)
45 try:
46     from skimage.filters import threshold_multiotsu
47     thresholds = threshold_multiotsu(seg_img, classes=4)
48     regions = np.digitize(seg_img, bins=thresholds)
49     multi_otsu = (regions * 85).astype(np.uint8) # Scale for visualization
50 except:
51     multi_otsu = seg_img.copy()
52
53 # Display thresholding results
54 fig, axes = plt.subplots(3, 4, figsize=(16, 12))
55
56 threshold_images = [
57     ('Original', seg_img, 'gray'),
58     ('Binary (127)', thresh_binary, 'gray'),
59     ('Binary Inv', thresh_binary_inv, 'gray'),
60     ('Trunc', thresh_trunc, 'gray'),
61     ('To Zero', thresh_tozero, 'gray'),
62     ('To Zero Inv', thresh_tozero_inv, 'gray'),
63     ('Adaptive Mean', thresh_mean, 'gray'),
64     ('Adaptive Gaussian', thresh_gaussian, 'gray'),
65     ("Otsu's", thresh_otsu, 'gray'),
66     ('Multi-Otsu (4 classes)', multi_otsu, 'gray'),
67     ('Histogram', seg_img, 'hist'),
68     ('Otsu Threshold', seg_img, 'hist_otsu')
69 ]
70
71 for idx, (title, img, cmap) in enumerate(threshold_images):
72     row, col = divmod(idx, 4)
73     if cmap == 'hist':
74         axes[row, col].hist(seg_img.ravel(), 256, [0, 256])
75         axes[row, col].set_title(title)
76         axes[row, col].set_xlabel('Pixel Value')

```

```

77     axes[row, col].set_ylabel('Frequency')
78     axes[row, col].grid(True, alpha=0.3)
79     elif cmap == 'hist_otsu':
80         axes[row, col].hist(seg_img.ravel(), 256, [0, 256])
81         axes[row, col].axvline(x=127, color='r', linestyle='--', label='Simple
            ↳ (127)')
82         otsu_thresh_val = cv2.threshold(seg_img, 0, 255, cv2.THRESH_BINARY + cv2.
            ↳ THRESH_OTSU)[0]
83         axes[row, col].axvline(x=otsu_thresh_val, color='g', linestyle='-', label=
            ↳ f'Otsu ({otsu_thresh_val:.0f})")
84         axes[row, col].set_title('Thresholds on Histogram')
85         axes[row, col].set_xlabel('Pixel Value')
86         axes[row, col].set_ylabel('Frequency')
87         axes[row, col].legend()
88         axes[row, col].grid(True, alpha=0.3)
89     else:
90         axes[row, col].imshow(img, cmap=cmap)
91         axes[row, col].set_title(title)
92         axes[row, col].axis('off')
93
94 plt.tight_layout()
95 plt.show()
96
97 # 4. Watershed Algorithm
98 # Create markers for watershed
99 # Convert to color for watershed
100 seg_img_color = cv2.cvtColor(seg_img, cv2.COLOR_GRAY2BGR)
101
102 # Apply Otsu's thresholding to get binary image
103 _, binary = cv2.threshold(seg_img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
104
105 # Noise removal
106 kernel = np.ones((3,3), np.uint8)
107 opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel, iterations=2)
108
109 # Sure background area
110 sure_bg = cv2.dilate(opening, kernel, iterations=3)
111
112 # Finding sure foreground area
113 dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
114 _, sure_fg = cv2.threshold(dist_transform, 0.5 * dist_transform.max(), 255, 0)
115
116 # Finding unknown region
117 sure_fg = np.uint8(sure_fg)
118 unknown = cv2.subtract(sure_bg, sure_fg)
119
120 # Marker labelling

```

```

121 _, markers = cv2.connectedComponents(sure_fg)
122
123 # Add one to all labels so that sure background is not 0, but 1
124 markers = markers + 1
125
126 # Mark the region of unknown with zero
127 markers[unknown == 255] = 0
128
129 # Apply watershed
130 watershed_result = seg_img_color.copy()
131 cv2.watershed(watershed_result, markers)
132 watershed_result[markers == -1] = [255, 0, 0] # Mark boundaries in red
133
134 # 5. GrabCut Algorithm (simulated since we need color image)
135 # Create a color version for GrabCut
136 color_img = np.zeros((200, 300, 3), dtype=np.uint8)
137 for i in range(3):
138     color_img[:, :, i] = seg_img
139
140 # Define rectangle (x, y, w, h) - roughly around the circles
141 rect = (50, 50, 200, 150)
142
143 # Create mask
144 mask = np.zeros(color_img.shape[:2], np.uint8)
145
146 # Initialize background and foreground models
147 bgd_model = np.zeros((1, 65), np.float64)
148 fgd_model = np.zeros((1, 65), np.float64)
149
150 # Apply GrabCut
151 cv2.grabCut(color_img, mask, rect, bgd_model, fgd_model, 5, cv2.GC_INIT_WITH_RECT
    ↪ )
152
153 # Create mask where sure or likely foreground
154 mask2 = np.where((mask == 2) | (mask == 0), 0, 1).astype('uint8')
155 grabcut_result = color_img * mask2[:, :, np.newaxis]
156
157 # 6. K-means Clustering for Segmentation
158 # Reshape image to 2D array of pixels
159 pixels = seg_img.reshape((-1, 1))
160 pixels = np.float32(pixels)
161
162 # Define criteria and apply k-means
163 criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)
164 k = 4 # Number of clusters
165 _, labels, centers = cv2.kmeans(pixels, k, None, criteria, 10, cv2.
    ↪ KMEANS_RANDOM_CENTERS)

```

```

166
167 # Convert back to 8-bit values
168 centers = np.uint8(centers)
169 kmeans_result = centers[labels.flatten()]
170 kmeans_result = kmeans_result.reshape(seg_img.shape)
171
172 # 7. Mean Shift Segmentation
173 # Reshape to feature vector (x, y, intensity)
174 height, width = seg_img.shape
175 y_coords, x_coords = np.mgrid[0:height, 0:width]
176 coords = np.column_stack([x_coords.ravel(), y_coords.ravel(), seg_img.ravel()])
177
178 # Apply mean shift (simplified version)
179 from sklearn.cluster import MeanShift, estimate_bandwidth
180
181 # Estimate bandwidth
182 bandwidth = estimate_bandwidth(coords[::10], quantile=0.2) # Sample for speed
183
184 # Apply Mean Shift
185 ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)
186 ms.fit(coords[::10]) # Fit on sample
187 labels_ms = ms.predict(coords)
188 mean_shift_result = labels_ms.reshape(seg_img.shape)
189 mean_shift_result = (mean_shift_result * (255 / mean_shift_result.max())).astype(
    ↪ np.uint8)
190
191 # Display segmentation results
192 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
193
194 segmentation_images = [
195     ('Original', seg_img, 'gray'),
196     ('Binary (Otsu)', binary, 'gray'),
197     ('Watershed', watershed_result, None),
198     ('GrabCut', grabcut_result, None),
199     ('K-means (k=4)', kmeans_result, 'gray'),
200     ('Mean Shift', mean_shift_result, 'gray')
201 ]
202
203 for idx, (title, img, cmap) in enumerate(segmentation_images):
204     row, col = divmod(idx, 3)
205     if cmap:
206         axes[row, col].imshow(img, cmap=cmap)
207     else:
208         axes[row, col].imshow(img)
209     axes[row, col].set_title(title)
210     axes[row, col].axis('off')
211

```

```

212 plt.tight_layout()
213 plt.show()
214
215 # 8. Region-based Segmentation
216 # Region Growing (simple implementation)
217 def region_growing(img, seed, threshold):
218     """Simple region growing algorithm."""
219     height, width = img.shape
220     visited = np.zeros((height, width), dtype=bool)
221     output = np.zeros((height, width), dtype=np.uint8)
222
223     # Define 4-connectivity
224     directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
225
226     # Initialize stack with seed
227     stack = [seed]
228     seed_value = img[seed]
229
230     while stack:
231         x, y = stack.pop()
232
233         if visited[x, y]:
234             continue
235
236         visited[x, y] = True
237
238         # Check if pixel is similar to seed
239         if abs(int(img[x, y]) - int(seed_value)) <= threshold:
240             output[x, y] = 255
241
242             # Add neighbors to stack
243             for dx, dy in directions:
244                 nx, ny = x + dx, y + dy
245                 if 0 <= nx < height and 0 <= ny < width and not visited[nx, ny]:
246                     stack.append((nx, ny))
247
248     return output
249
250 # Apply region growing from different seeds
251 seed1 = (75, 75) # Center of dark circle
252 seed2 = (150, 150) # Center of bright circle
253 threshold = 30
254
255 region1 = region_growing(seg_img, seed1, threshold)
256 region2 = region_growing(seg_img, seed2, threshold)
257
258 # Combine regions

```

```

259 combined_regions = cv2.add(region1, region2)
260
261 # 9. Contour-based Segmentation
262 # Find contours in binary image
263 contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    ↪ )
264
265 # Create contour-based segmentation
266 contour_seg = np.zeros_like(seg_img)
267 for i, contour in enumerate(contours):
268     # Fill each contour with different intensity
269     cv2.drawContours(contour_seg, [contour], -1, (i+1) * 50, -1)
270
271 # 10. Edge-based Segmentation
272 # Use Canny edges
273 edges = cv2.Canny(seg_img, 50, 150)
274
275 # Dilate edges to close gaps
276 kernel = np.ones((3,3), np.uint8)
277 dilated_edges = cv2.dilate(edges, kernel, iterations=1)
278
279 # Fill regions
280 edge_seg = dilated_edges.copy()
281 edge_seg = cv2.morphologyEx(edge_seg, cv2.MORPH_CLOSE, kernel, iterations=2)
282
283 # Find contours and fill
284 contours, _ = cv2.findContours(edge_seg, cv2.RETR_EXTERNAL, cv2.
    ↪ CHAIN_APPROX_SIMPLE)
285 edge_filled = np.zeros_like(seg_img)
286 for i, contour in enumerate(contours):
287     cv2.drawContours(edge_filled, [contour], -1, (i+1) * 60, -1)
288
289 # Display region-based segmentation results
290 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
291
292 region_images = [
293     ('Region Growing Seed1', region1, 'gray'),
294     ('Region Growing Seed2', region2, 'gray'),
295     ('Combined Regions', combined_regions, 'gray'),
296     ('Contour-based', contour_seg, 'gray'),
297     ('Canny Edges', edges, 'gray'),
298     ('Edge-based', edge_filled, 'gray')
299 ]
300
301 for idx, (title, img, cmap) in enumerate(region_images):
302     row, col = divmod(idx, 3)
303     axes[row, col].imshow(img, cmap=cmap)

```

```

304     axes[row, col].set_title(title)
305     axes[row, col].axis('off')
306
307 plt.tight_layout()
308 plt.show()
309
310 # Segmentation evaluation metrics (for comparison with ground truth)
311 # Create a simple ground truth
312 ground_truth = np.zeros_like(seg_img)
313 cv2.circle(ground_truth, (75, 75), 40, 255, -1)
314 cv2.circle(ground_truth, (225, 75), 40, 255, -1)
315 cv2.circle(ground_truth, (150, 150), 40, 255, -1)
316
317 # Binarize segmentation results for evaluation
318 _, binary_gt = cv2.threshold(ground_truth, 127, 255, cv2.THRESH_BINARY)
319 _, binary_seg = cv2.threshold(binary, 127, 255, cv2.THRESH_BINARY)
320
321 # Calculate metrics
322 def calculate_metrics(gt, pred):
323     """Calculate segmentation evaluation metrics."""
324     # True positives, false positives, true negatives, false negatives
325     tp = np.sum((gt == 255) & (pred == 255))
326     fp = np.sum((gt == 0) & (pred == 255))
327     tn = np.sum((gt == 0) & (pred == 0))
328     fn = np.sum((gt == 255) & (pred == 0))
329
330     # Calculate metrics
331     accuracy = (tp + tn) / (tp + tn + fp + fn) if (tp + tn + fp + fn) > 0 else 0
332     precision = tp / (tp + fp) if (tp + fp) > 0 else 0
333     recall = tp / (tp + fn) if (tp + fn) > 0 else 0
334     f1_score = 2 * precision * recall / (precision + recall) if (precision +
335         ↪ recall) > 0 else 0
336     iou = tp / (tp + fp + fn) if (tp + fp + fn) > 0 else 0
337
338     return {
339         'accuracy': accuracy,
340         'precision': precision,
341         'recall': recall,
342         'f1_score': f1_score,
343         'iou': iou
344     }
345
346 metrics = calculate_metrics(binary_gt, binary_seg)
347
348 print("Segmentation Evaluation Metrics:")
349 print("-" * 50)
350 print(f"Accuracy: {metrics['accuracy']:.4f}")

```

```

350 print(f"Precision: {metrics['precision']:.4f}")
351 print(f"Recall: {metrics['recall']:.4f}")
352 print(f"F1-Score: {metrics['f1_score']:.4f}")
353 print(f"IoU: {metrics['iou']:.4f}")
354 print("-" * 50)
355
356 # Visualize ground truth vs segmentation
357 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
358
359 comparison_images = [
360     ('Ground Truth', ground_truth, 'gray'),
361     ('Segmentation Result', binary, 'gray'),
362     ('Overlay', cv2.addWeighted(ground_truth, 0.5, binary, 0.5, 0), 'gray')
363 ]
364
365 for idx, (title, img, cmap) in enumerate(comparison_images):
366     axes[idx].imshow(img, cmap=cmap)
367     axes[idx].set_title(title)
368     axes[idx].axis('off')
369
370 plt.tight_layout()
371 plt.show()

```

Listing 6: Image Segmentation Techniques

7 Convolutional Neural Networks (CNNs)

7.1 The Convolution Operation in Neural Networks

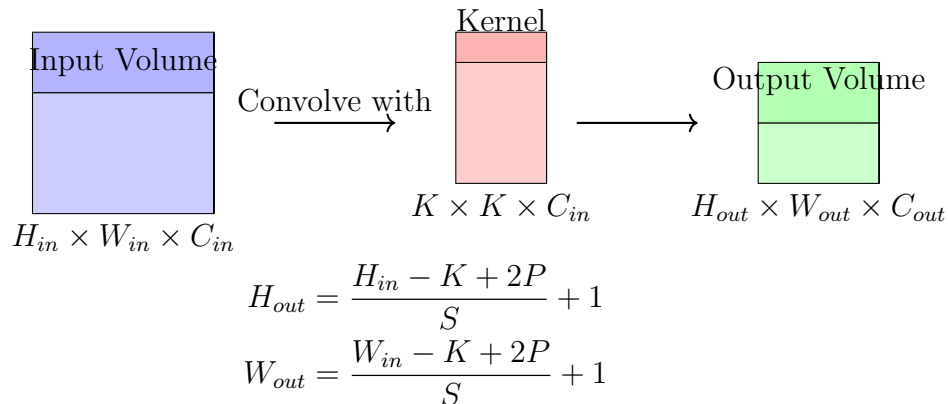


Figure 11: 3D convolution in CNNs

7.2 CNN Architecture Components

Layer Type	Purpose	Key Parameters
Convolution	Extract features using filters	Filter size, Stride, Padding, Number of filters
Pooling	Reduce spatial dimensions	Pool size, Stride (Max/Average pooling)
Activation	Introduce non-linearity	ReLU, Sigmoid, Tanh
Normalization	Stabilize training	BatchNorm, LayerNorm
Fully Connected	Final classification/regression	Number of neurons
Dropout	Prevent overfitting	Dropout rate

Table 8: Components of CNN architecture

7.3 Famous CNN Architectures

Architecture	Key Innovation	Depth	Top-5 Error (ImageNet)
AlexNet (2012)	First deep CNN to win ImageNet	8 layers	15.3%
VGG (2014)	Simple 3×3 convolutions	16/19 layers	7.3%
GoogLeNet (2014)	Inception modules	22 layers	6.7%
ResNet (2015)	Residual connections	50/101/152 layers	3.57%
DenseNet (2017)	Dense connections	121/201 layers	3.46%
EfficientNet (2019)	Compound scaling	B0-B7 variants	2.36%

Table 9: Evolution of CNN architectures

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import torch.optim as optim
5 from torch.utils.data import DataLoader, TensorDataset
6 import numpy as np
7 import matplotlib.pyplot as plt
8
9 # Set random seeds for reproducibility

```

```

10 torch.manual_seed(42)
11 np.random.seed(42)
12
13 # Create synthetic image dataset
14 def create_synthetic_dataset(n_samples=1000, img_size=32):
15     """Create synthetic dataset of simple shapes."""
16     images = []
17     labels = []
18
19     for i in range(n_samples):
20         img = np.zeros((img_size, img_size), dtype=np.float32)
21
22         # Randomly choose shape
23         label = np.random.randint(0, 3) # 0: circle, 1: square, 2: triangle
24
25         if label == 0: # Circle
26             center = (np.random.randint(10, img_size-10),
27                       np.random.randint(10, img_size-10))
28             radius = np.random.randint(5, 10)
29             cv2.circle(img, center, radius, 1.0, -1)
30
31         elif label == 1: # Square
32             x = np.random.randint(5, img_size-15)
33             y = np.random.randint(5, img_size-15)
34             size = np.random.randint(8, 15)
35             cv2.rectangle(img, (x, y), (x+size, y+size), 1.0, -1)
36
37         else: # Triangle
38             pt1 = (np.random.randint(5, img_size-5), np.random.randint(5, img_size
39                 ↳ //2))
40             pt2 = (pt1[0] - np.random.randint(5, 10), pt1[1] + np.random.randint
41                 ↳ (10, 15))
42             pt3 = (pt1[0] + np.random.randint(5, 10), pt1[1] + np.random.randint
43                 ↳ (10, 15))
44             pts = np.array([pt1, pt2, pt3], np.int32)
45             cv2.fillPoly(img, [pts], 1.0)
46
47         # Add noise
48         noise = np.random.normal(0, 0.1, (img_size, img_size)).astype(np.float32)
49         img = img + noise
50         img = np.clip(img, 0, 1)
51
52         # Add channel dimension
53         img = img.reshape(1, img_size, img_size)
54
55         images.append(img)
56         labels.append(label)

```

```

54
55     return np.array(images), np.array(labels)
56
57 # Create dataset
58 X, y = create_synthetic_dataset(1000, 32)
59
60 # Split into train and test
61 train_size = int(0.8 * len(X))
62 X_train, X_test = X[:train_size], X[train_size:]
63 y_train, y_test = y[:train_size], y[train_size:]
64
65 # Convert to PyTorch tensors
66 X_train_tensor = torch.FloatTensor(X_train)
67 y_train_tensor = torch.LongTensor(y_train)
68 X_test_tensor = torch.FloatTensor(X_test)
69 y_test_tensor = torch.LongTensor(y_test)
70
71 # Create datasets and dataloaders
72 train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
73 test_dataset = TensorDataset(X_test_tensor, y_test_tensor)
74
75 train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
76 test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
77
78 # 1. Simple CNN Architecture
79 class SimpleCNN(nn.Module):
80     def __init__(self, num_classes=3):
81         super(SimpleCNN, self).__init__()
82
83         # Convolutional layers
84         self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1) # 32x32 -> 32x32
85         self.bn1 = nn.BatchNorm2d(16)
86         self.pool1 = nn.MaxPool2d(2) # 32x32 -> 16x16
87
88         self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1) # 16x16 -> 16x16
89         self.bn2 = nn.BatchNorm2d(32)
90         self.pool2 = nn.MaxPool2d(2) # 16x16 -> 8x8
91
92         self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1) # 8x8 -> 8x8
93         self.bn3 = nn.BatchNorm2d(64)
94         self.pool3 = nn.MaxPool2d(2) # 8x8 -> 4x4
95
96         # Fully connected layers
97         self.fc1 = nn.Linear(64 * 4 * 4, 128)
98         self.dropout1 = nn.Dropout(0.5)
99         self.fc2 = nn.Linear(128, num_classes)
100

```

```

101     def forward(self, x):
102         # Convolutional blocks
103         x = self.pool1(F.relu(self.bn1(self.conv1(x))))
104         x = self.pool2(F.relu(self.bn2(self.conv2(x))))
105         x = self.pool3(F.relu(self.bn3(self.conv3(x))))
106
107         # Flatten
108         x = x.view(x.size(0), -1)
109
110         # Fully connected
111         x = F.relu(self.fc1(x))
112         x = self.dropout1(x)
113         x = self.fc2(x)
114
115         return x
116
117 # 2. More Complex CNN with Advanced Features
118 class AdvancedCNN(nn.Module):
119     def __init__(self, num_classes=3):
120         super(AdvancedCNN, self).__init__()
121
122         # First block: Conv -> BatchNorm -> ReLU -> Conv -> BatchNorm -> ReLU ->
123         ↪ MaxPool
124         self.block1 = nn.Sequential(
125             nn.Conv2d(1, 32, kernel_size=3, padding=1),
126             nn.BatchNorm2d(32),
127             nn.ReLU(),
128             nn.Conv2d(32, 32, kernel_size=3, padding=1),
129             nn.BatchNorm2d(32),
130             nn.ReLU(),
131             nn.MaxPool2d(2),
132             nn.Dropout2d(0.25)
133         )
134
135         # Second block
136         self.block2 = nn.Sequential(
137             nn.Conv2d(32, 64, kernel_size=3, padding=1),
138             nn.BatchNorm2d(64),
139             nn.ReLU(),
140             nn.Conv2d(64, 64, kernel_size=3, padding=1),
141             nn.BatchNorm2d(64),
142             nn.ReLU(),
143             nn.MaxPool2d(2),
144             nn.Dropout2d(0.25)
145         )
146
147         # Third block

```

```

147     self.block3 = nn.Sequential(
148         nn.Conv2d(64, 128, kernel_size=3, padding=1),
149         nn.BatchNorm2d(128),
150         nn.ReLU(),
151         nn.Conv2d(128, 128, kernel_size=3, padding=1),
152         nn.BatchNorm2d(128),
153         nn.ReLU(),
154         nn.MaxPool2d(2),
155         nn.Dropout2d(0.25)
156     )
157
158     # Fully connected layers
159     self.fc = nn.Sequential(
160         nn.Linear(128 * 4 * 4, 256),
161         nn.ReLU(),
162         nn.Dropout(0.5),
163         nn.Linear(256, num_classes)
164     )
165
166     def forward(self, x):
167         x = self.block1(x)
168         x = self.block2(x)
169         x = self.block3(x)
170         x = x.view(x.size(0), -1)
171         x = self.fc(x)
172         return x
173
174     # 3. Residual Block (for ResNet-like architecture)
175     class ResidualBlock(nn.Module):
176         def __init__(self, in_channels, out_channels, stride=1):
177             super(ResidualBlock, self).__init__()
178
179             self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
180                                     stride=stride, padding=1, bias=False)
181             self.bn1 = nn.BatchNorm2d(out_channels)
182             self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
183                                     stride=1, padding=1, bias=False)
184             self.bn2 = nn.BatchNorm2d(out_channels)
185
186             # Shortcut connection
187             self.shortcut = nn.Sequential()
188             if stride != 1 or in_channels != out_channels:
189                 self.shortcut = nn.Sequential(
190                     nn.Conv2d(in_channels, out_channels, kernel_size=1,
191                             stride=stride, bias=False),
192                     nn.BatchNorm2d(out_channels)
193                 )

```

```

194
195     def forward(self, x):
196         residual = x
197
198         out = F.relu(self.bn1(self.conv1(x)))
199         out = self.bn2(self.conv2(out))
200
201         out += self.shortcut(residual)
202         out = F.relu(out)
203
204         return out
205
206 # 4. ResNet-like CNN
207 class ResNetCNN(nn.Module):
208     def __init__(self, num_classes=3):
209         super(ResNetCNN, self).__init__()
210
211         self.in_channels = 16
212
213         # Initial convolution
214         self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1, bias=False)
215         self.bn1 = nn.BatchNorm2d(16)
216
217         # Residual blocks
218         self.layer1 = self._make_layer(16, 2, stride=1)
219         self.layer2 = self._make_layer(32, 2, stride=2)
220         self.layer3 = self._make_layer(64, 2, stride=2)
221
222         # Global average pooling
223         self.avg_pool = nn.AdaptiveAvgPool2d((1, 1))
224
225         # Fully connected
226         self.fc = nn.Linear(64, num_classes)
227
228     def _make_layer(self, out_channels, blocks, stride):
229         strides = [stride] + [1] * (blocks - 1)
230         layers = []
231
232         for stride in strides:
233             layers.append(ResidualBlock(self.in_channels, out_channels, stride))
234             self.in_channels = out_channels
235
236         return nn.Sequential(*layers)
237
238     def forward(self, x):
239         x = F.relu(self.bn1(self.conv1(x)))
240

```

```

241         x = self.layer1(x)
242         x = self.layer2(x)
243         x = self.layer3(x)
244
245         x = self.avg_pool(x)
246         x = x.view(x.size(0), -1)
247         x = self.fc(x)
248
249         return x
250
251     # Initialize models
252     simple_cnn = SimpleCNN(num_classes=3)
253     advanced_cnn = AdvancedCNN(num_classes=3)
254     resnet_cnn = ResNetCNN(num_classes=3)
255
256     print("Simple CNN Architecture:")
257     print(simple_cnn)
258     print("\n" + "="*50)
259
260     print("\nAdvanced CNN Architecture:")
261     print(advanced_cnn)
262     print("\n" + "="*50)
263
264     # Training function
265     def train_model(model, train_loader, test_loader, num_epochs=20, lr=0.001):
266         """Train a CNN model."""
267         device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
268         model = model.to(device)
269
270         criterion = nn.CrossEntropyLoss()
271         optimizer = optim.Adam(model.parameters(), lr=lr)
272         scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
273
274         train_losses = []
275         train_accuracies = []
276         test_accuracies = []
277
278         for epoch in range(num_epochs):
279             # Training phase
280             model.train()
281             running_loss = 0.0
282             correct = 0
283             total = 0
284
285             for batch_idx, (inputs, targets) in enumerate(train_loader):
286                 inputs, targets = inputs.to(device), targets.to(device)
287

```

```

288     optimizer.zero_grad()
289     outputs = model(inputs)
290     loss = criterion(outputs, targets)
291     loss.backward()
292     optimizer.step()
293
294     running_loss += loss.item()
295     _, predicted = outputs.max(1)
296     total += targets.size(0)
297     correct += predicted.eq(targets).sum().item()
298
299     train_loss = running_loss / len(train_loader)
300     train_accuracy = 100. * correct / total
301
302     # Testing phase
303     model.eval()
304     test_correct = 0
305     test_total = 0
306
307     with torch.no_grad():
308         for inputs, targets in test_loader:
309             inputs, targets = inputs.to(device), targets.to(device)
310             outputs = model(inputs)
311             _, predicted = outputs.max(1)
312             test_total += targets.size(0)
313             test_correct += predicted.eq(targets).sum().item()
314
315     test_accuracy = 100. * test_correct / test_total
316
317     # Update scheduler
318     scheduler.step()
319
320     # Store metrics
321     train_losses.append(train_loss)
322     train_accuracies.append(train_accuracy)
323     test_accuracies.append(test_accuracy)
324
325     if (epoch + 1) % 5 == 0:
326         print(f'Epoch [{epoch+1}/{num_epochs}], '
327               f'Loss: {train_loss:.4f}, '
328               f'Train Acc: {train_accuracy:.2f}%, '
329               f'Test Acc: {test_accuracy:.2f}%')
330
331     return train_losses, train_accuracies, test_accuracies
332
333 # Train Simple CNN
334 print("\nTraining Simple CNN...")

```

```

335 train_losses_simple, train_acc_simple, test_acc_simple = train_model(
336     simple_cnn, train_loader, test_loader, num_epochs=30
337 )
338
339 # Train Advanced CNN
340 print("\nTraining Advanced CNN...")
341 train_losses_adv, train_acc_adv, test_acc_adv = train_model(
342     advanced_cnn, train_loader, test_loader, num_epochs=30
343 )
344
345 # Plot training results
346 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
347
348 # Loss curves
349 axes[0, 0].plot(train_losses_simple, label='Simple CNN')
350 axes[0, 0].plot(train_losses_adv, label='Advanced CNN')
351 axes[0, 0].set_xlabel('Epoch')
352 axes[0, 0].set_ylabel('Loss')
353 axes[0, 0].set_title('Training Loss')
354 axes[0, 0].legend()
355 axes[0, 0].grid(True, alpha=0.3)
356
357 # Accuracy curves
358 axes[0, 1].plot(train_acc_simple, label='Simple CNN Train')
359 axes[0, 1].plot(test_acc_simple, label='Simple CNN Test')
360 axes[0, 1].set_xlabel('Epoch')
361 axes[0, 1].set_ylabel('Accuracy (%)')
362 axes[0, 1].set_title('Simple CNN Accuracy')
363 axes[0, 1].legend()
364 axes[0, 1].grid(True, alpha=0.3)
365
366 axes[0, 2].plot(train_acc_adv, label='Advanced CNN Train')
367 axes[0, 2].plot(test_acc_adv, label='Advanced CNN Test')
368 axes[0, 2].set_xlabel('Epoch')
369 axes[0, 2].set_ylabel('Accuracy (%)')
370 axes[0, 2].set_title('Advanced CNN Accuracy')
371 axes[0, 2].legend()
372 axes[0, 2].grid(True, alpha=0.3)
373
374 # Visualize filters
375 def visualize_filters(model, layer_name):
376     """Visualize convolutional filters."""
377     filters = None
378     for name, param in model.named_parameters():
379         if layer_name in name and 'weight' in name and param.dim() == 4:
380             filters = param.data.cpu().numpy()
381             break

```

```

382
383     if filters is None:
384         return
385
386     n_filters = min(filters.shape[0], 16)
387     fig, axes = plt.subplots(4, 4, figsize=(10, 10))
388
389     for i in range(n_filters):
390         row, col = divmod(i, 4)
391         filter_img = filters[i, 0, :, :] # First channel
392         axes[row, col].imshow(filter_img, cmap='gray')
393         axes[row, col].axis('off')
394         axes[row, col].set_title(f'Filter {i+1}')
395
396     plt.suptitle(f'{layer_name} Filters')
397     plt.tight_layout()
398     plt.show()
399
400 # Visualize filters from first conv layer
401 visualize_filters(simple_cnn, 'conv1')
402 visualize_filters(advanced_cnn, 'block1.0')
403
404 # Feature maps visualization
405 def visualize_feature_maps(model, sample_image):
406     """Visualize feature maps at different layers."""
407     model.eval()
408
409     # Hook to capture feature maps
410     feature_maps = {}
411
412     def get_feature_maps(name):
413         def hook(model, input, output):
414             feature_maps[name] = output.detach()
415         return hook
416
417     # Register hooks
418     hooks = []
419     for name, layer in model.named_modules():
420         if isinstance(layer, nn.Conv2d):
421             hook = layer.register_forward_hook(get_feature_maps(name))
422             hooks.append(hook)
423
424     # Forward pass
425     with torch.no_grad():
426         _ = model(sample_image.unsqueeze(0))
427
428     # Remove hooks

```

```

429     for hook in hooks:
430         hook.remove()
431
432     # Visualize feature maps
433     for layer_name, maps in feature_maps.items():
434         maps = maps.squeeze(0).cpu().numpy()
435         n_maps = min(maps.shape[0], 16)
436
437         fig, axes = plt.subplots(4, 4, figsize=(12, 12))
438         fig.suptitle(f'Feature Maps: {layer_name}', fontsize=16)
439
440         for i in range(n_maps):
441             row, col = divmod(i, 4)
442             axes[row, col].imshow(maps[i], cmap='viridis')
443             axes[row, col].axis('off')
444             axes[row, col].set_title(f'Map {i+1}')
445
446         plt.tight_layout()
447         plt.show()
448
449     # Visualize feature maps for a sample
450     sample_idx = 0
451     sample_image = X_test_tensor[sample_idx]
452     visualize_feature_maps(simple_cnn, sample_image)
453
454     # CNN interpretability: Grad-CAM
455     class GradCAM:
456         def __init__(self, model, target_layer):
457             self.model = model
458             self.target_layer = target_layer
459             self.gradients = None
460             self.activations = None
461
462             # Register hooks
463             target_layer.register_forward_hook(self.save_activations)
464             target_layer.register_backward_hook(self.save_gradients)
465
466         def save_activations(self, module, input, output):
467             self.activations = output.detach()
468
469         def save_gradients(self, module, grad_input, grad_output):
470             self.gradients = grad_output[0].detach()
471
472         def __call__(self, x, class_idx=None):
473             # Forward pass
474             output = self.model(x)
475

```

```

476         if class_idx is None:
477             class_idx = output.argmax(dim=1).item()
478
479         # Backward pass for specific class
480         self.model.zero_grad()
481         one_hot = torch.zeros_like(output)
482         one_hot[0, class_idx] = 1
483         output.backward(gradient=one_hot)
484
485         # Calculate weights
486         weights = self.gradients.mean(dim=(2, 3), keepdim=True)
487
488         # Calculate CAM
489         cam = (weights * self.activations).sum(dim=1, keepdim=True)
490         cam = F.relu(cam)
491         cam = F.interpolate(cam, size=x.shape[2:], mode='bilinear', align_corners=
            ↪ False)
492
493         # Normalize
494         cam_min, cam_max = cam.min(), cam.max()
495         cam = (cam - cam_min) / (cam_max - cam_min + 1e-8)
496
497         return cam.squeeze().cpu().numpy()
498
499     # Apply Grad-CAM
500     gradcam = GradCAM(simple_cnn, simple_cnn.conv3)
501     cam = gradcam(sample_image.unsqueeze(0))
502
503     # Visualize Grad-CAM
504     fig, axes = plt.subplots(1, 3, figsize=(12, 4))
505
506     axes[0].imshow(sample_image.squeeze(), cmap='gray')
507     axes[0].set_title('Original Image')
508     axes[0].axis('off')
509
510     axes[1].imshow(cam, cmap='jet')
511     axes[1].set_title('Grad-CAM')
512     axes[1].axis('off')
513
514     # Overlay
515     heatmap = cv2.applyColorMap(np.uint8(255 * cam), cv2.COLORMAP_JET)
516     heatmap = cv2.cvtColor(heatmap, cv2.COLOR_BGR2RGB)
517     original = (sample_image.squeeze().numpy() * 255).astype(np.uint8)
518     original = cv2.cvtColor(original, cv2.COLOR_GRAY2RGB)
519     overlay = cv2.addWeighted(original, 0.6, heatmap, 0.4, 0)
520
521     axes[2].imshow(overlay)

```

```

522 axes[2].set_title('Overlay')
523 axes[2].axis('off')
524
525 plt.tight_layout()
526 plt.show()
527
528 # Test model on sample images
529 def test_sample_images(model, sample_images, sample_labels):
530     """Test model on sample images."""
531     model.eval()
532     device = next(model.parameters()).device
533
534     with torch.no_grad():
535         sample_images = sample_images.to(device)
536         outputs = model(sample_images)
537         _, predicted = outputs.max(1)
538
539     # Visualize predictions
540     n_samples = min(len(sample_images), 9)
541     fig, axes = plt.subplots(3, 3, figsize=(10, 10))
542
543     for i in range(n_samples):
544         row, col = divmod(i, 3)
545         img = sample_images[i].cpu().squeeze().numpy()
546         true_label = sample_labels[i].item()
547         pred_label = predicted[i].item()
548
549         axes[row, col].imshow(img, cmap='gray')
550         axes[row, col].axis('off')
551
552         color = 'green' if true_label == pred_label else 'red'
553         axes[row, col].set_title(f'True: {true_label}, Pred: {pred_label}', color=
            ↪ color)
554
555     plt.suptitle('Model Predictions on Test Samples', fontsize=16)
556     plt.tight_layout()
557     plt.show()
558
559 # Test on random samples
560 sample_indices = np.random.choice(len(X_test_tensor), 9, replace=False)
561 sample_imgs = X_test_tensor[sample_indices]
562 sample_labs = y_test_tensor[sample_indices]
563
564 test_sample_images(simple_cnn, sample_imgs, sample_labs)
565
566 # Model evaluation metrics
567 def evaluate_model_comprehensive(model, test_loader):

```

```

568     """Comprehensive model evaluation."""
569     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
570     model = model.to(device)
571     model.eval()
572
573     all_preds = []
574     all_labels = []
575
576     with torch.no_grad():
577         for inputs, labels in test_loader:
578             inputs, labels = inputs.to(device), labels.to(device)
579             outputs = model(inputs)
580             _, preds = outputs.max(1)
581
582             all_preds.extend(preds.cpu().numpy())
583             all_labels.extend(labels.cpu().numpy())
584
585     # Calculate metrics
586     from sklearn.metrics import classification_report, confusion_matrix,
587         ↪ accuracy_score
588
589     accuracy = accuracy_score(all_labels, all_preds)
590     cm = confusion_matrix(all_labels, all_preds)
591     report = classification_report(all_labels, all_preds,
592                                   target_names=['Circle', 'Square', 'Triangle'])
593
594     print(f"Overall Accuracy: {accuracy:.4f}")
595     print("\nConfusion Matrix:")
596     print(cm)
597     print("\nClassification Report:")
598     print(report)
599
600     return accuracy, cm, report
601
602 print("\nSimple CNN Evaluation:")
603 print("="*50)
604 evaluate_model_comprehensive(simple_cnn, test_loader)
605
606 print("\nAdvanced CNN Evaluation:")
607 print("="*50)
608 evaluate_model_comprehensive(advanced_cnn, test_loader)

```

Listing 7: CNN Implementation with PyTorch

8 Advanced Computer Vision Topics

8.1 Object Detection

Method	Approach	Architecture	Speed vs Accuracy
R-CNN	Region proposals + CNN	Selective Search + CNN	Slow, High accuracy
Fast R-CNN	Shared features	ROI Pooling	Faster
Faster R-CNN	Neural proposals	RPN + Fast R-CNN	Real-time capable
YOLO	Single pass detection	Grid-based prediction	Very fast, Good accuracy
SSD	Multi-scale feature maps	Default boxes + CNN	Fast, Good for small objects
RetinaNet	Focal loss for imbalance	Feature Pyramid Network	Best for small objects

Table 10: Object detection algorithms comparison

8.2 Image Segmentation (Advanced)

Method	Type	Architecture	Use Case
FCN	Semantic	Fully Convolutional Network	Scene understanding
U-Net	Biomedical	Encoder-Decoder with skip	Medical imaging
Mask R-CNN	Instance	Faster R-CNN + Mask branch	Object segmentation
DeepLab	Semantic	Atrous convolution + ASPP	High-resolution
PSPNet	Semantic	Pyramid Pooling Module	Scene parsing

Table 11: Advanced image segmentation methods

8.3 GANs for Computer Vision

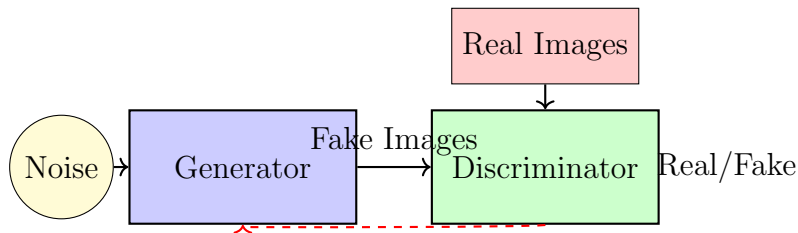


Figure 12: GAN architecture: Generator creates fake images, Discriminator tries to distinguish real from fake

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import torchvision
5 import torchvision.transforms as transforms
6 import numpy as np
7 import matplotlib.pyplot as plt
8
9 # 1. Object Detection with YOLO-like architecture
10 class YOLOLikeDetector(nn.Module):
11     def __init__(self, grid_size=7, num_boxes=2, num_classes=20):
12         super(YOLOLikeDetector, self).__init__()
13         self.grid_size = grid_size
14         self.num_boxes = num_boxes
15         self.num_classes = num_classes
16
17     # CNN backbone
18     self.backbone = nn.Sequential(
19         # Block 1
20         nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3),
21         nn.BatchNorm2d(64),
22         nn.LeakyReLU(0.1),
23         nn.MaxPool2d(2, stride=2),
24
25         # Block 2
26         nn.Conv2d(64, 192, kernel_size=3, padding=1),
27         nn.BatchNorm2d(192),
28         nn.LeakyReLU(0.1),
29         nn.MaxPool2d(2, stride=2),
30
31         # Block 3
32         nn.Conv2d(192, 128, kernel_size=1),
33         nn.Conv2d(128, 256, kernel_size=3, padding=1),
34         nn.Conv2d(256, 256, kernel_size=1),
```

```

35         nn.Conv2d(256, 512, kernel_size=3, padding=1),
36         nn.BatchNorm2d(512),
37         nn.LeakyReLU(0.1),
38         nn.MaxPool2d(2, stride=2),
39     )
40
41     # Detection head
42     self.detection_head = nn.Sequential(
43         nn.Conv2d(512, 1024, kernel_size=3, padding=1),
44         nn.BatchNorm2d(1024),
45         nn.LeakyReLU(0.1),
46
47         nn.Conv2d(1024, 512, kernel_size=3, padding=1),
48         nn.BatchNorm2d(512),
49         nn.LeakyReLU(0.1),
50
51         nn.Conv2d(512, (5 * num_boxes + num_classes), kernel_size=1),
52     )
53
54     def forward(self, x):
55         # Backbone features
56         features = self.backbone(x)
57
58         # Detection predictions
59         predictions = self.detection_head(features)
60
61         # Reshape to (batch, grid, grid, boxes*(5+classes))
62         batch_size = predictions.size(0)
63         predictions = predictions.permute(0, 2, 3, 1).contiguous()
64         predictions = predictions.view(batch_size, self.grid_size, self.grid_size,
65                                       self.num_boxes * 5 + self.num_classes)
66
67         return predictions
68
69     # 2. Semantic Segmentation with U-Net
70     class UNet(nn.Module):
71         def __init__(self, in_channels=3, out_channels=1, features=[64, 128, 256,
72             ↪ 512]):
73             super(UNet, self).__init__()
74
75             # Encoder (downsampling)
76             self.encoder = nn.ModuleList()
77             self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
78
79             for feature in features:
80                 self.encoder.append(UNet._block(in_channels, feature))
81                 in_channels = feature

```

```

81
82     # Bottleneck
83     self.bottleneck = UNet._block(features[-1], features[-1] * 2)
84
85     # Decoder (upsampling)
86     self.decoder = nn.ModuleList()
87     self.upconvs = nn.ModuleList()
88
89     for feature in reversed(features):
90         self.upconvs.append(
91             nn.ConvTranspose2d(feature * 2, feature, kernel_size=2, stride=2)
92         )
93         self.decoder.append(UNet._block(feature * 2, feature))
94
95     # Final convolution
96     self.final_conv = nn.Conv2d(features[0], out_channels, kernel_size=1)
97
98     @staticmethod
99     def _block(in_channels, out_channels):
100         return nn.Sequential(
101             nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1, bias=
102                 ↪ False),
103             nn.BatchNorm2d(out_channels),
104             nn.ReLU(inplace=True),
105             nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, bias=
106                 ↪ False),
107             nn.BatchNorm2d(out_channels),
108             nn.ReLU(inplace=True)
109         )
110
111     def forward(self, x):
112         # Encoder
113         skip_connections = []
114
115         for down in self.encoder:
116             x = down(x)
117             skip_connections.append(x)
118             x = self.pool(x)
119
120         # Bottleneck
121         x = self.bottleneck(x)
122         skip_connections = skip_connections[::-1] # Reverse for decoder
123
124         # Decoder
125         for idx, (upconv, decoder) in enumerate(zip(self.upconvs, self.decoder)):
126             x = upconv(x)

```

```

126         # Skip connection
127         skip_connection = skip_connections[idx]
128
129         # Adjust dimensions if necessary
130         if x.shape != skip_connection.shape:
131             x = F.interpolate(x, size=skip_connection.shape[2:], mode='bilinear
132                               ↪ ', align_corners=True)
133
134         x = torch.cat([skip_connection, x], dim=1)
135         x = decoder(x)
136
137         # Final convolution
138         return self.final_conv(x)
139
140 # 3. Generative Adversarial Network (GAN)
141 class Generator(nn.Module):
142     def __init__(self, latent_dim=100, img_channels=3):
143         super(Generator, self).__init__()
144
145         self.init_size = 32 // 4 # Initial size before upsampling
146         self.l1 = nn.Sequential(
147             nn.Linear(latent_dim, 128 * self.init_size ** 2)
148         )
149
150         self.conv_blocks = nn.Sequential(
151             nn.BatchNorm2d(128),
152             nn.Upsample(scale_factor=2),
153             nn.Conv2d(128, 128, 3, stride=1, padding=1),
154             nn.BatchNorm2d(128, 0.8),
155             nn.LeakyReLU(0.2, inplace=True),
156
157             nn.Upsample(scale_factor=2),
158             nn.Conv2d(128, 64, 3, stride=1, padding=1),
159             nn.BatchNorm2d(64, 0.8),
160             nn.LeakyReLU(0.2, inplace=True),
161
162             nn.Conv2d(64, img_channels, 3, stride=1, padding=1),
163             nn.Tanh()
164         )
165
166     def forward(self, z):
167         out = self.l1(z)
168         out = out.view(out.shape[0], 128, self.init_size, self.init_size)
169         img = self.conv_blocks(out)
170         return img
171
172 class Discriminator(nn.Module):

```

```

172 def __init__(self, img_channels=3):
173     super(Discriminator, self).__init__()
174
175     def discriminator_block(in_filters, out_filters, bn=True):
176         block = [nn.Conv2d(in_filters, out_filters, 3, 2, 1),
177                 nn.LeakyReLU(0.2, inplace=True),
178                 nn.Dropout2d(0.25)]
179         if bn:
180             block.append(nn.BatchNorm2d(out_filters, 0.8))
181         return block
182
183     self.model = nn.Sequential(
184         *discriminator_block(img_channels, 16, bn=False),
185         *discriminator_block(16, 32),
186         *discriminator_block(32, 64),
187         *discriminator_block(64, 128),
188     )
189
190     # The height and width of downsampled image
191     ds_size = 32 // 2 ** 4
192     self.adv_layer = nn.Sequential(
193         nn.Linear(128 * ds_size ** 2, 1),
194         nn.Sigmoid()
195     )
196
197     def forward(self, img):
198         out = self.model(img)
199         out = out.view(out.shape[0], -1)
200         validity = self.adv_layer(out)
201         return validity
202
203 # 4. Vision Transformer (ViT) Simplified
204 class PatchEmbedding(nn.Module):
205     def __init__(self, img_size=224, patch_size=16, in_channels=3, embed_dim=768):
206         super().__init__()
207         self.img_size = img_size
208         self.patch_size = patch_size
209         self.n_patches = (img_size // patch_size) ** 2
210
211         self.proj = nn.Conv2d(in_channels, embed_dim,
212                               kernel_size=patch_size, stride=patch_size)
213
214     def forward(self, x):
215         x = self.proj(x) # (B, E, H/P, W/P)
216         x = x.flatten(2) # (B, E, N)
217         x = x.transpose(1, 2) # (B, N, E)
218         return x

```

```

219
220 class VisionTransformer(nn.Module):
221     def __init__(self, img_size=224, patch_size=16, in_channels=3,
222                 num_classes=1000, embed_dim=768, depth=12, num_heads=12):
223         super().__init__()
224
225         self.patch_embed = PatchEmbedding(img_size, patch_size, in_channels,
226                                           ↪ embed_dim)
227         num_patches = self.patch_embed.n_patches
228
229         # CLS token
230         self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
231
232         # Position embedding
233         self.pos_embed = nn.Parameter(torch.zeros(1, num_patches + 1, embed_dim))
234
235         # Transformer encoder
236         encoder_layer = nn.TransformerEncoderLayer(
237             d_model=embed_dim, nhead=num_heads, dim_feedforward=embed_dim * 4
238         )
239         self.transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers
240                                                         ↪ =depth)
241
242         # Classifier head
243         self.head = nn.Linear(embed_dim, num_classes)
244
245         self._init_weights()
246
247     def _init_weights(self):
248         nn.init.trunc_normal_(self.pos_embed, std=0.02)
249         nn.init.trunc_normal_(self.cls_token, std=0.02)
250
251     def forward(self, x):
252         B = x.shape[0]
253
254         # Patch embedding
255         x = self.patch_embed(x)
256
257         # Add CLS token
258         cls_tokens = self.cls_token.expand(B, -1, -1)
259         x = torch.cat((cls_tokens, x), dim=1)
260
261         # Add position embedding
262         x = x + self.pos_embed
263
264         # Transformer encoder
265         x = self.transformer_encoder(x)

```

```

264
265     # CLS token output
266     x = x[:, 0]
267
268     # Classification head
269     x = self.head(x)
270
271     return x
272
273 # 5. Attention Mechanism for Computer Vision
274 class SpatialAttention(nn.Module):
275     def __init__(self, kernel_size=7):
276         super(SpatialAttention, self).__init__()
277
278         assert kernel_size in (3, 7), 'kernel size must be 3 or 7'
279         padding = 3 if kernel_size == 7 else 1
280
281         self.conv1 = nn.Conv2d(2, 1, kernel_size, padding=padding, bias=False)
282         self.sigmoid = nn.Sigmoid()
283
284     def forward(self, x):
285         avg_out = torch.mean(x, dim=1, keepdim=True)
286         max_out, _ = torch.max(x, dim=1, keepdim=True)
287         combined = torch.cat([avg_out, max_out], dim=1)
288         attention = self.sigmoid(self.conv1(combined))
289         return x * attention
290
291 class ChannelAttention(nn.Module):
292     def __init__(self, in_channels, reduction_ratio=16):
293         super(ChannelAttention, self).__init__()
294
295         self.avg_pool = nn.AdaptiveAvgPool2d(1)
296         self.max_pool = nn.AdaptiveMaxPool2d(1)
297
298         self.fc = nn.Sequential(
299             nn.Conv2d(in_channels, in_channels // reduction_ratio, 1, bias=False),
300             nn.ReLU(),
301             nn.Conv2d(in_channels // reduction_ratio, in_channels, 1, bias=False)
302         )
303
304         self.sigmoid = nn.Sigmoid()
305
306     def forward(self, x):
307         avg_out = self.fc(self.avg_pool(x))
308         max_out = self.fc(self.max_pool(x))
309         out = avg_out + max_out
310         return x * self.sigmoid(out)

```

```

311
312 class CBAMBlock(nn.Module):
313     def __init__(self, in_channels, reduction_ratio=16, kernel_size=7):
314         super(CBAMBlock, self).__init__()
315
316         self.channel_attention = ChannelAttention(in_channels, reduction_ratio)
317         self.spatial_attention = SpatialAttention(kernel_size)
318
319     def forward(self, x):
320         x = self.channel_attention(x)
321         x = self.spatial_attention(x)
322         return x
323
324 # Demonstration and testing
325 def test_models():
326     """Test different computer vision models."""
327
328     # Test input
329     test_input = torch.randn(1, 3, 224, 224)
330
331     print("Testing Different Computer Vision Models:")
332     print("=" * 60)
333
334     # Test YOLO-like detector
335     yolo = YOLOLikeDetector(grid_size=7, num_boxes=2, num_classes=20)
336     yolo_output = yolo(test_input[:, :, :224, :224]) # Resize if needed
337     print(f"YOLO-like Detector output shape: {yolo_output.shape}")
338
339     # Test U-Net
340     unet = UNet(in_channels=3, out_channels=1)
341     unet_output = unet(test_input)
342     print(f"U-Net output shape: {unet_output.shape}")
343
344     # Test Generator
345     latent_dim = 100
346     generator = Generator(latent_dim=latent_dim)
347     noise = torch.randn(1, latent_dim)
348     gen_output = generator(noise)
349     print(f"Generator output shape: {gen_output.shape}")
350
351     # Test Discriminator
352     discriminator = Discriminator()
353     disc_output = discriminator(gen_output)
354     print(f"Discriminator output shape: {disc_output.shape}")
355
356     # Test Vision Transformer (small version)
357     vit = VisionTransformer(img_size=224, patch_size=16, num_classes=1000,

```

```

358         embed_dim=192, depth=6, num_heads=3)
359 vit_output = vit(test_input)
360 print(f"Vision Transformer output shape: {vit_output.shape}")
361
362 # Test CBAM
363 cbam = CBAMBlock(in_channels=64)
364 test_feature = torch.randn(1, 64, 56, 56)
365 cbam_output = cbam(test_feature)
366 print(f"CBAM output shape: {cbam_output.shape}")
367
368 print("=" * 60)
369 print("All models tested successfully!")
370
371 # Run tests
372 test_models()
373
374 # Visualization functions
375 def visualize_model_architectures():
376     """Visualize different model architectures."""
377     fig, axes = plt.subplots(2, 3, figsize=(15, 10))
378
379     # YOLO visualization
380     axes[0, 0].text(0.5, 0.5, 'YOLO Architecture\n\n'
381                    'Grid-based detection\n'
382                    'Single forward pass\n'
383                    'Bounding box + class\n'
384                    'Fast inference',
385                    ha='center', va='center', fontsize=10,
386                    bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.5))
387     axes[0, 0].set_title('YOLO Object Detection')
388     axes[0, 0].axis('off')
389
390     # U-Net visualization
391     axes[0, 1].text(0.5, 0.5, 'U-Net Architecture\n\n'
392                    'Encoder-Decoder\n'
393                    'Skip connections\n'
394                    'Medical imaging\n'
395                    'Pixel-wise segmentation',
396                    ha='center', va='center', fontsize=10,
397                    bbox=dict(boxstyle='round', facecolor='lightgreen', alpha=0.5))
398     axes[0, 1].set_title('U-Net Segmentation')
399     axes[0, 1].axis('off')
400
401     # GAN visualization
402     axes[0, 2].text(0.5, 0.5, 'GAN Architecture\n\n'
403                    'Generator + Discriminator\n'
404                    'Adversarial training\n')

```

```

405         ' Image generation\n'
406         ' Style transfer',
407         ha='center', va='center', fontsize=10,
408         bbox=dict(boxstyle='round', facecolor='lightcoral', alpha=0.5))
409 axes[0, 2].set_title('Generative Adversarial Network')
410 axes[0, 2].axis('off')
411
412 # Vision Transformer visualization
413 axes[1, 0].text(0.5, 0.5, 'Vision Transformer\n\n'
414         ' Patch-based processing\n'
415         ' Self-attention\n'
416         ' Global context\n'
417         ' State-of-the-art',
418         ha='center', va='center', fontsize=10,
419         bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.5)
420         ↪ )
421 axes[1, 0].set_title('Vision Transformer (ViT)')
422 axes[1, 0].axis('off')
423
424 # Attention visualization
425 axes[1, 1].text(0.5, 0.5, 'Attention Mechanism\n\n'
426         ' Channel attention\n'
427         ' Spatial attention\n'
428         ' CBAM blocks\n'
429         ' Feature refinement',
430         ha='center', va='center', fontsize=10,
431         bbox=dict(boxstyle='round', facecolor='lightpurple', alpha=0.5)
432         ↪ )
433 axes[1, 1].set_title('Attention in CV')
434 axes[1, 1].axis('off')
435
436 # Modern architectures
437 axes[1, 2].text(0.5, 0.5, 'Modern Architectures\n\n'
438         ' EfficientNet\n'
439         ' DETR\n'
440         ' Swin Transformer\n'
441         ' ConvNeXt',
442         ha='center', va='center', fontsize=10,
443         bbox=dict(boxstyle='round', facecolor='lightgray', alpha=0.5))
444 axes[1, 2].set_title('Modern CV Models')
445 axes[1, 2].axis('off')
446
447 plt.suptitle('Advanced Computer Vision Architectures', fontsize=16)
448 plt.tight_layout()
449 plt.show()
450
451 visualize_model_architectures()

```

```

450
451 # Transfer learning example
452 def transfer_learning_example():
453     """Example of transfer learning with pre-trained models."""
454
455     # Load pre-trained ResNet
456     resnet = torchvision.models.resnet18(pretrained=True)
457
458     # Freeze all layers
459     for param in resnet.parameters():
460         param.requires_grad = False
461
462     # Replace the final layer for our task
463     num_features = resnet.fc.in_features
464     resnet.fc = nn.Sequential(
465         nn.Linear(num_features, 256),
466         nn.ReLU(),
467         nn.Dropout(0.5),
468         nn.Linear(256, 10) # 10 classes for CIFAR-10
469     )
470
471     print("Transfer Learning with ResNet18:")
472     print(" Pre-trained on ImageNet")
473     print(" Frozen feature extractor")
474     print(" Custom classifier for 10 classes")
475     print(f" Total parameters: {sum(p.numel() for p in resnet.parameters())}")
476     print(f" Trainable parameters: {sum(p.numel() for p in resnet.parameters() if
477         ↪ p.requires_grad)}")
478
479     return resnet
480
481 model = transfer_learning_example()
482
483 # Data augmentation pipeline
484 def create_augmentation_pipeline():
485     """Create data augmentation pipeline for training."""
486
487     train_transform = transforms.Compose([
488         transforms.RandomResizedCrop(224),
489         transforms.RandomHorizontalFlip(),
490         transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue
491             ↪ =0.1),
492         transforms.RandomRotation(15),
493         transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)),
494         transforms.ToTensor(),
495         transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
496             ↪ 0.225])

```

```

494     ])
495
496     test_transform = transforms.Compose([
497         transforms.Resize(256),
498         transforms.CenterCrop(224),
499         transforms.ToTensor(),
500         transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
501             ↪ 0.225])
502     ])
503
504     print("\nData Augmentation Pipeline:")
505     print("Training augmentations:")
506     print(" Random crop and resize")
507     print(" Random horizontal flip")
508     print(" Color jittering")
509     print(" Random rotation")
510     print(" Random translation")
511     print(" Normalization (ImageNet stats)")
512
513     return train_transform, test_transform
514
515 train_transform, test_transform = create_augmentation_pipeline()
516
517 # Model deployment considerations
518 def model_deployment_considerations():
519     """Discuss model deployment considerations."""
520
521     considerations = {
522         'Inference Speed': [
523             'Model size and complexity',
524             'Hardware acceleration (GPU/TPU)',
525             'Batch size optimization',
526             'Model quantization'
527         ],
528         'Memory Usage': [
529             'Model parameter count',
530             'Activation memory',
531             'Batch size impact',
532             'Mixed precision training'
533         ],
534         'Deployment Options': [
535             'Edge devices (mobile, embedded)',
536             'Cloud deployment',
537             'Real-time vs batch processing',
538             'Model serving frameworks'
539         ],
540         'Optimization Techniques': [

```

```

540         'Pruning',
541         'Quantization',
542         'Knowledge distillation',
543         'Neural architecture search'
544     ]
545 }
546
547 print("\n" + "="*60)
548 print("Model Deployment Considerations")
549 print("="*60)
550
551 for category, items in considerations.items():
552     print(f"\n{category}:")
553     for item in items:
554         print(f" {item}")
555
556 model_deployment_considerations()
557
558 # Performance metrics for different tasks
559 def performance_metrics():
560     """Discuss performance metrics for different CV tasks."""
561
562     metrics = {
563         'Classification': {
564             'Accuracy': 'Overall correctness',
565             'Precision/Recall': 'Class-wise performance',
566             'F1-Score': 'Harmonic mean',
567             'Confusion Matrix': 'Error analysis',
568             'ROC-AUC': 'Threshold-independent'
569         },
570         'Object Detection': {
571             'mAP': 'Mean Average Precision',
572             'IoU': 'Intersection over Union',
573             'Precision-Recall Curve': 'Trade-off analysis',
574             'FPS': 'Frames per second'
575         },
576         'Segmentation': {
577             'Pixel Accuracy': 'Per-pixel correctness',
578             'IoU (Jaccard Index)': 'Overlap measure',
579             'Dice Coefficient': 'Similarity measure',
580             'Per-class metrics': 'Class-wise performance'
581         },
582         'Generation': {
583             'FID': 'Frechet Inception Distance',
584             'IS': 'Inception Score',
585             'PSNR': 'Peak Signal-to-Noise Ratio',
586             'SSIM': 'Structural Similarity'

```

```

587     }
588 }
589
590 print("\n" + "="*60)
591 print("Performance Metrics for Computer Vision Tasks")
592 print("="*60)
593
594 for task, task_metrics in metrics.items():
595     print(f"\n{task}:")
596     for metric, description in task_metrics.items():
597         print(f"  {metric}: {description}")
598
599 performance_metrics()

```

Listing 8: Advanced Computer Vision with PyTorch

9 Computer Vision Applications

9.1 Face Recognition

1. **Face Detection:** Find faces in images (Haar cascades, MTCNN)
2. **Face Alignment:** Normalize face orientation
3. **Feature Extraction:** Extract facial features (DeepFace, FaceNet)
4. **Face Matching:** Compare with database

9.2 Optical Character Recognition (OCR)

1. **Text Detection:** Find text regions in images
2. **Text Recognition:** Convert image to text (Tesseract, EasyOCR)
3. **Post-processing:** Correct recognition errors

9.3 Medical Imaging

- **X-ray analysis:** Pneumonia detection, fracture detection
- **MRI/CT segmentation:** Tumor detection, organ segmentation
- **Microscopy:** Cell counting, disease diagnosis

10 Best Practices and Future Directions

10.1 Best Practices in Computer Vision

Practice	Description	Benefit
Data Augmentation	Apply transformations to increase dataset diversity	Prevents overfitting
Transfer Learning	Use pre-trained models as starting point	Faster training, Better performance
Model Ensemble	Combine multiple models	Improved accuracy and robustness
Cross-validation	Validate on multiple splits	Reliable performance estimate
Regularization	Apply dropout, weight decay	Prevents overfitting
Learning Rate Schedule	Adjust LR during training	Better convergence
Early Stopping	Stop when validation performance plateaus	Prevents overfitting
Model Interpretability	Use Grad-CAM, attention maps	Understand model decisions

Table 12: Best practices in computer vision

10.2 Emerging Trends

1. **Vision Transformers:** Self-attention for images
2. **Self-supervised Learning:** Learning from unlabeled data
3. **Multimodal Learning:** Combining vision with text, audio
4. **Neural Rendering:** Generating 3D from 2D images
5. **Few-shot Learning:** Learning from few examples
6. **Explainable AI:** Making models interpretable
7. **Efficient Models:** Lightweight models for edge devices
8. **3D Computer Vision:** Depth estimation, point clouds

10.3 Resources and Tools

Tool/Library	Purpose	Key Features
OpenCV	Computer vision library	Image processing, Feature detection
PyTorch	Deep learning framework	Dynamic computation graphs
TensorFlow	Deep learning framework	Production deployment
scikit-image	Image processing	Algorithms, filters
Albumentations	Data augmentation	Fast augmentations
MMDetection	Object detection	Multiple detection algorithms
DETR	Detection with Transformers	End-to-end object detection
YOLO	Real-time detection	Fast inference

Table 13: Tools and libraries for computer vision

11 Conclusion

Key Takeaways

1. **Start with fundamentals:** Understand pixels, color spaces, basic operations
2. **Master classical methods:** Learn filtering, edge detection, feature extraction
3. **Progress to deep learning:** CNNs are fundamental for modern computer vision
4. **Choose right architecture:** Different tasks need different models
5. **Focus on data quality:** Good data is more important than complex models
6. **Consider deployment:** Model size and speed matter in production
7. **Stay updated:** Computer vision is rapidly evolving

11.1 Next Steps

1. **Specialize:** Choose an area (medical imaging, autonomous vehicles, etc.)
2. **Build portfolio:** Work on projects and share on GitHub
3. **Learn 3D vision:** Depth estimation, point cloud processing
4. **Explore video analysis:** Temporal understanding, action recognition

5. **Study multimodal AI:** Combine vision with language, audio
6. **Learn deployment:** Model optimization, serving, monitoring

11.2 Final Advice

”Computer vision is not just about algorithms; it’s about understanding the visual world. Always ask: What problem am I solving? Who will use this? What are the ethical implications?”

Ethical Considerations

- **Bias:** Models can inherit biases from training data
- **Privacy:** Face recognition and surveillance raise privacy concerns
- **Security:** Adversarial attacks can fool vision systems
- **Transparency:** Users should understand how decisions are made
- **Accountability:** Clear responsibility for system failures

Keep Learning: Computer vision is one of the most exciting fields in AI. Stay curious, keep experimenting, and contribute to making technology that benefits everyone.